

DEEP LEARNING, TOLD SIMPLY

Book 5 of 6

WRITTEN BY

Prof. Saurabh

A Real Brain

Hidden layers, activation functions, backpropagation by hand, and how to design a network that learns

One neuron hit a wall at XOR. Asha adds a hidden layer and watches it bend the picture until one straight line is enough. Then she trains a tiny 2-3-2 network by hand — forward, loss, backward, update — and Nova learns to hear. Module 5 and its Numerical deck, step by step, in plain words.



What is inside

Book 5 covers Module 5: Deep Feed-Forward Neural Networks, together with the Module 5 Numerical deck. Eight chapters. Chapter 5 is the heart of the book: one complete training step done by hand, with a picture of the network at every step.

1	The wall, and the way over it	3
	One neuron can only draw a straight line. XOR needs a bend. A hidden layer redraws the picture so that one line is enough again.	
2	Bending the line: activation functions	8
	A layer without a bend is just a line, and two lines in a row are still one line. The bend is the activation function. Here are the three you must know, with their slopes.	
3	The four parts of a deep network	14
	Data, model, loss, learning algorithm — the same four parts as in Book 1. The model is now a stack of layers, so we need a clean way to name every layer, unit and weight.	
4	Forward, loss, backward	19
	The forward pass makes predictions and keeps notes. The loss scores them. The backward pass sends one error signal back through the layers and turns it into every gradient at once.	
5	By hand: one whole training step	24
	A 2-3-2 network, one input, one target. Forward, loss, backward, update — every number worked out on the whiteboard, with a picture of the network at every step.	
6	Gradient flow, depth, and designing a network	34
	Why deep sigmoid networks stop learning, why depth is still worth it, and how to choose the number of layers and units for a dataset.	
7	The training toolbox	40
	How to start the weights, which knobs to set, how to prepare the data, what to watch while training, what to do when it goes wrong — and how a trained network answers a new question.	
8	Losses, their slopes, and what comes next	45
	Softmax with a worked problem, then every loss you have met with its derivative, side by side. Then the takeaways of both decks, and where Nova goes from here.	

How to read this book

Each chapter follows the same rhythm: a little story, one big idea, an everyday picture of it, the maths in small steps (first plain, then in matrix form), a worked example with real numbers, and a short check. The coloured boxes tell you what kind of thing you are reading:

- | | |
|--|--|
|  Story — Asha and Nova, the thread through all six books |  Big idea — the one thing to remember |
|  Think of it like... — a real-life picture of the idea |  Stop and think — a question, then Our answer |
|  Maths, step by step — plain maths → matrix form |  Worked example — real numbers, one line per step |
|  On your slides — the exact notation your deck uses |  Careful — common mistakes |
|  Spark — a deeper idea for the curious |  Check yourself — quick questions with answers |

The wall, and the way over it

One neuron can only draw a straight line. XOR needs a bend. A hidden layer redraws the picture so that one line is enough again.

On your slides: **Module 5, slides 4–8** · Linear models have limitations · The XOR problem · Why a single neuron fails · Multi-layer perceptron · MLP solves XOR

🔥 STORY

The hostel corridor has one lamp and two switches, one at each end. Flip either switch and the lamp changes. So the lamp is on when *exactly one* switch is up. Asha wants Nova to copy that lamp: two inputs, one light. She trains a single neuron, the way she did in Book 4. It learns three of the four cases and never the fourth. Ravi leans in the doorway. "You told me in Book 2 that one cell cannot do this." Asha nods. "So tonight I give Nova a second layer."

! BIG IDEA

A single neuron decides with one straight line. Some problems, like XOR, cannot be cut by any straight line. The fix is a **hidden layer**: a few neurons in the middle that rewrite the inputs as new features. In the new picture the classes *can* be cut by one line. Stacking layers with a bend (a non-linear activation) after each one is called a **multi-layer perceptron**.

≈ THINK OF IT LIKE...

Two orange dots and two blue dots drawn on a sheet of paper, at the corners of a square, opposite corners the same colour. You are allowed one straight cut with scissors. You cannot separate the colours. Now *fold* the paper first, so the two orange corners land on top of each other. One straight cut does it. The fold is the hidden layer. The cut is the output neuron. The fold does not cut anything by itself; it only moves the dots to better places.

Linear models hit a wall

In Books 3 and 4 every model was one neuron: add up the weighted inputs, then apply one activation f .

$$\hat{y} = f(\mathbf{w}^\top \mathbf{x} + b)$$

Here $\mathbf{x}$ is the input, $\mathbf{w}$ the weights, b the bias, and f the activation (identity, sigmoid or softmax). This worked for house prices and for pass/fail. But most real relationships are not straight-line relationships. Your slide lists four:

- Pixel intensities → object class. A cat can be anywhere in the picture, in any pose.
- Words → sentiment. "Not bad" is positive, "not good" is negative: the word *not* flips things.
- Symptoms → disease probability. Two mild symptoms together can mean more than each one alone.
- Board state → best move. The value of a piece depends on where every other piece is.

Your slide says it in one line: **linear models cannot solve non-linearly separable problems**. Let us look at the smallest such problem.

The XOR problem

XOR means "exclusive or": output 1 when exactly one input is 1. It is the corridor lamp.

x_1	x_2	y (XOR)
0	0	0
0	1	1
1	0	1
1	1	0

Draw the four rows as points. The two 1s sit at opposite corners, and so do the two 0s. Try to draw a straight line with both orange points on one side and both blue points on the other. You cannot. Every line you try leaves one point on the wrong side.

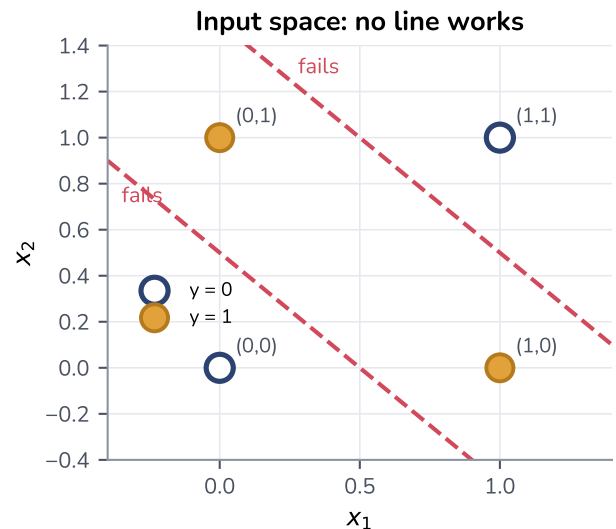


Figure 1.1 The XOR points. Orange means $y = 1$, blue means $y = 0$. The two dashed lines are the two best tries; each one gets three points right and one wrong. Notice that the orange points are on opposite corners: no single straight line can keep them together.

Why a single neuron fails

Take one neuron with two inputs: $\hat{y} = \sigma(w_1x_1 + w_2x_2 + b)$. Your slide says σ can be *any* activation. The neuron says "class 1" when its input sum $z = w_1x_1 + w_2x_2 + b$ is above some value, and "class 0" below it. The border between the two answers is where z equals that value:

$$w_1x_1 + w_2x_2 + b = 0$$

That is the equation of a straight line in the (x_1, x_2) plane (a flat **hyperplane** in more dimensions). Whatever σ is, the border stays a line, because σ only changes what happens *after* the sum. And Figure 1.1 shows no line works. Your slide's conclusion: **we need models that can learn non-linear decision boundaries**.

? STOP AND THINK

Ravi asks: "What if I pick a cleverer activation, one with a bump in the middle? Would one neuron solve XOR then?"

✓ OUR ANSWER

No. The output still depends on z only, so every point on the same line $z = c$ gets the same answer. The neuron can only tell points apart by which parallel line they sit on, and no set of parallel lines puts $(0, 1)$ and $(1, 0)$ together and $(0, 0)$ and $(1, 1)$ together. The real fix is not a better activation; it is a second layer.

The way over the wall: a hidden layer

Your slide's key idea: **stack multiple layers of neurons with non-linear activations**. Put two neurons between the inputs and the output. They are called the **hidden layer**, because you never see their values from outside. Each hidden neuron reads the inputs, makes its own sum, and applies a bend. The output neuron then reads the hidden neurons instead of the raw inputs. Your slide draws it as three columns: Input Layer, Hidden Layer, Output Layer, with every unit in one column connected to every unit in the next. Figure 1.2 is that picture with one working set of numbers written on it.

How the MLP solves XOR

Your slide gives four bullet points. Here they are with the corridor lamp in mind:

- **The hidden layer creates new features.** Instead of x_1 and x_2 , the output sees h_1 and h_2 .
- **Each hidden unit learns one straight line.** A hidden unit is a single neuron, so it can only draw a line. That is fine: we have two of them.
- **The output layer combines the lines.** It weighs h_1 against h_2 and decides.
- **The non-linear activation makes the combination bend.** Without it, two lines added together are just one line (Chapter 2 shows this).

Here is one set of weights that works. We use ReLU as the bend: $\text{ReLU}(z) = \max(0, z)$, which keeps positive numbers and turns negatives into 0.

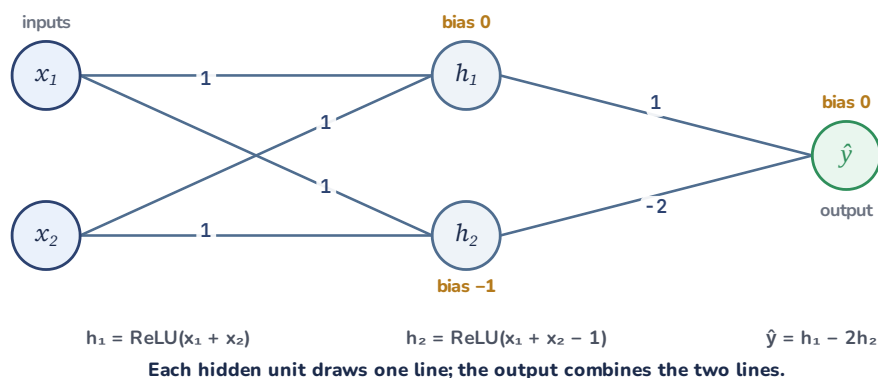


Figure 1.2 A 2-2-1 network that solves XOR. Hidden unit 1 adds the inputs. Hidden unit 2 adds the inputs and subtracts 1, so it only "switches on" when both inputs are 1. The output rewards h_1 and punishes h_2 twice as hard.

WORKED EXAMPLE: ALL FOUR INPUTS THROUGH THE NETWORK

Run each row of the XOR table through the three formulas. One line per number.

- 1 $\mathbf{x} = (0, 0)$: $h_1 = \text{ReLU}(0) = 0$, $h_2 = \text{ReLU}(-1) = 0$, $\hat{y} = 0 - 0 = 0$.
ReLU turns -1 into 0 — class 0, correct
- 2 $\mathbf{x} = (0, 1)$: $h_1 = \text{ReLU}(1) = 1$, $h_2 = \text{ReLU}(0) = 0$, $\hat{y} = 1 - 0 = 1$. Class 1, correct.
- 3 $\mathbf{x} = (1, 0)$: $h_1 = \text{ReLU}(1) = 1$, $h_2 = \text{ReLU}(0) = 0$, $\hat{y} = 1 - 0 = 1$. Same hidden values as row 2.
- 4 $\mathbf{x} = (1, 1)$: $h_1 = \text{ReLU}(2) = 2$, $h_2 = \text{ReLU}(1) = 1$, $\hat{y} = 2 - 2 = 0$.
 h_2 wakes up only here and cancels h_1 — class 0, correct

All four right. Now write the same three formulas in matrix form, the way every deep network is written.

Σ MATHS, STEP BY STEP

PLAIN MATHS

Hidden unit 1: weights 1 and 1, bias 0, then
ReLU: $h_1 = \text{ReLU}(x_1 + x_2)$.

Hidden unit 2: weights 1 and 1, bias -1 , then
ReLU: $h_2 = \text{ReLU}(x_1 + x_2 - 1)$.

Output: weight 1 on h_1 , weight -2 on h_2 , bias 0: $\hat{y} = h_1 - 2h_2$.

Say "class 1" when $\hat{y} > 0.5$. Three sums, two bends, one cut.

MATRIX FORM

Write the input as a row $\mathbf{x} = (x_1, x_2)$, size 1×2 . The hidden weights are a 2×2 matrix and the hidden biases a 1×2 row:

$$\mathbf{h} = \text{ReLU}(\mathbf{x}\mathbf{W}^{(1)} + \mathbf{b}^{(1)}), \quad \mathbf{W}^{(1)} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \quad \mathbf{b}^{(1)} = (0, -1)$$

$\mathbf{h}$ is 1×2 . The output weights are a 2×1 column:

$$\hat{y} = \mathbf{h}\mathbf{w}^{(2)}, \quad \mathbf{w}^{(2)} = \begin{pmatrix} 1 \\ -2 \end{pmatrix}$$

Same numbers as the left side, but one multiplication per layer. This is the shape every deep network has: multiply, add bias, bend, repeat.

Now look at where the four points landed in the hidden picture.

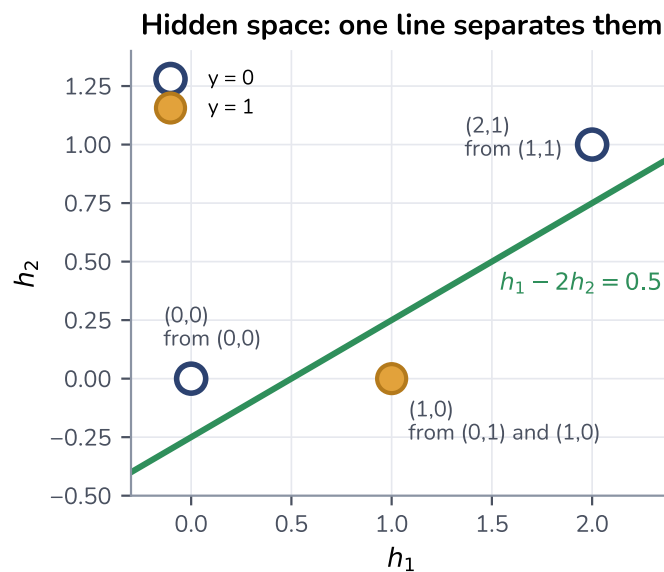


Figure 1.3 After the fold: hidden space (h_1, h_2) . Both orange points landed on the same spot (1,0). The blue points went to (0,0) and (2,1). The green line $h_1 - 2h_2 = 0.5$ now separates them. That green line *is* the output neuron.

Compare Figure 1.3 with Figure 1.1. Before, the orange points sat on opposite corners and no line could keep them together. After the hidden layer, they sit on the *same* spot, and one line is enough. This is the picture on your slide: *Input space* on the left, *Hidden space* on the right, and the word "Separable!".

The hidden layer folded the paper. Each hidden unit is still a plain neuron drawing a plain line; the fold comes from the ReLU bend, which pins every point below its line to zero. The output neuron then makes the one cut.

ON YOUR SLIDES

Slide 4 recalls $\hat{y} = f(\mathbf{w}^\top \mathbf{x} + b)$ and lists the four non-linear tasks. Slide 5 is the XOR truth table with the picture and the words "No linear separator!". Slide 6 writes the decision boundary $w_1x_1 + w_2x_2 + b = 0$ and shows two lines marked "Fails". Slide 7 is the MLP picture (Input Layer, Hidden Layer, Output Layer) with the caption "Non-linear activation functions enable non-linear boundaries". Slide 8, "MLP Solves XOR", has the four bullets and the Input Space $\rightarrow$ Hidden Space picture. The weights in Figure 1.2 are not on the slide; they are one working choice so you can see the fold with real numbers.

CAREFUL

The bend is not optional. Take the same network and remove the ReLUs: $h_1 = x_1 + x_2$, $h_2 = x_1 + x_2 - 1$, so $\hat{y} = h_1 - 2h_2 = 2 - x_1 - x_2$. That is a straight line again, and it gives 2, 1, 1, 0 for the four inputs instead of 0, 1, 1, 0. Two linear layers are still one linear layer. Chapter 2 makes this exact.

TRY IT YOURSELF

Change the output weights to $(1, -1)$, so $\hat{y} = h_1 - h_2$. Recompute the four outputs. Which input is now wrong? (Outputs 0, 1, 1, 1. The input $(1, 1)$ gives $2 - 1 = 1$; it should be 0. The weight -2 is needed so that h_2 can cancel the bigger h_1 .)

CHECK YOURSELF

1. What shape is the decision boundary of a single neuron with two inputs? **A straight line: $w_1x_1 + w_2x_2 + b = 0$.**
2. Why does changing the activation not help a single neuron with XOR? **The activation acts after the sum; the boundary is still where the sum is constant, a line.**
3. What does the hidden layer do to the XOR points? **It maps them to new features (h_1, h_2) where one line separates the classes.**
4. In the working network, which input makes h_2 non-zero? **Only $(1, 1)$: $h_2 = \text{ReLU}(1 + 1 - 1) = 1$.**
5. What is an MLP? **A multi-layer perceptron: layers of neurons stacked, with a non-linear activation after each layer.**

IN ONE BREATH

- One neuron = one straight line. XOR cannot be cut by a line.
- A hidden layer makes new features. Each hidden unit draws a line; the output combines them.
- In hidden space the XOR points become separable: the paper is folded, one cut is enough.
- The bend (activation) after each layer is what makes the fold. Without it, the layers collapse into one line.

Bending the line: activation functions

A layer without a bend is just a line, and two lines in a row are still one line. The bend is the activation function. Here are the three you must know, with their slopes.

On your slides: **Module 5, slides 10–13** · Non-linearity · Common activation functions · Properties · Why ReLU works well · **Numerical deck, slides 44–46, 50–56** · Sigmoid and tanh: definitions, derivatives, plots · Activation numerical problem

STORY

Asha builds Nova a network with two layers but, by mistake, no activation at all. Nova's tiny screen shows the same straight-line answers as before. Ravi grins: "Two layers, same mistake." Asha checks the maths: two multiplications in a row are one multiplication. The layers have collapsed. She needs a bend between them. There are three common bends. She wants to know what each one does, and how steep it is, because tomorrow she must send errors backwards through them.

BIG IDEA

A function is **non-linear** if it does not behave like a straight line. Without a non-linear activation, any number of layers collapses into a single linear layer. Sigmoid squashes to $(0, 1)$, tanh squashes to $(-1, 1)$, and ReLU keeps positives and zeroes negatives. Their **slopes** matter as much as their shapes: sigmoid's slope is at most 0.25, tanh's at most 1, ReLU's is exactly 1 for positive inputs. That is why ReLU is the default in hidden layers.

THINK OF IT LIKE...

A straight pipe is linear: push in twice the water, get out twice the water; join two pipes and you still have a pipe. A one-way valve is not: water flows forward, but nothing flows back. Join a pipe, a valve, another pipe, another valve, and the system can do things no single pipe can do. ReLU is that one-way valve: positive numbers pass, negative ones are stopped at zero.

What "non-linear" means

DEFINITION

A function f is **linear** if scaling and adding inputs is the same as scaling and adding outputs, for all numbers a, b and all inputs x_1, x_2 . A function is **non-linear** if this fails:

$$f(ax_1 + bx_2) \neq af(x_1) + bf(x_2)$$

Test it on two functions. Take $f(z) = 2z, a = b = 1, x_1 = 3, x_2 = 1$: $f(3 + 1) = 8$ and $f(3) + f(1) = 6 + 2 = 8$. Same. It is linear. Now $f(z) = \max(0, z)$ with $x_1 = -1, x_2 = 3$: $f(-1 + 3) = f(2) = 2$, but $f(-1) + f(3) = 0 + 3 = 3$. Different. ReLU is non-linear.

Why it matters: layers collapse without it

Suppose a layer has no activation. Then the layer is just "multiply by a matrix". Put two such layers in a row and you multiply by two matrices in a row, which is the same as multiplying by one matrix (their product). The second layer added nothing. Your slide says it in one line: **without non-linearity, deep networks collapse to single-layer models.**

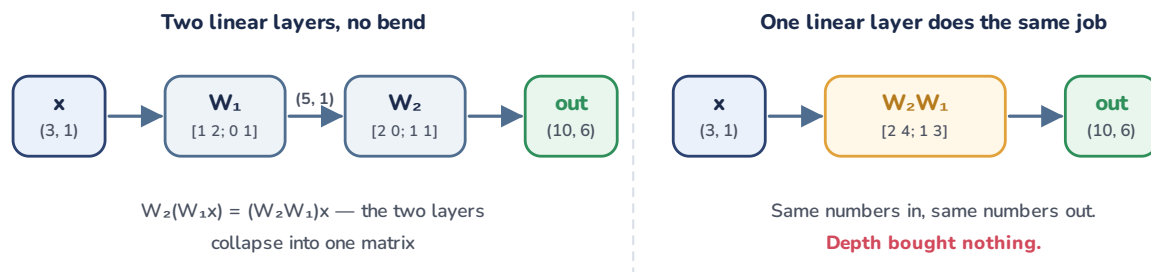


Figure 2.1 Left: the input $(3, 1)$ goes through $\mathbf{W}_1$ and then $\mathbf{W}_2$, giving $(10, 6)$. Right: multiply the two matrices first, $\mathbf{W}_2\mathbf{W}_1$, and one layer gives the same $(10, 6)$. Notice that no choice of the two matrices can escape this. Two lines in a row are one line.

Σ MATHS, STEP BY STEP

PLAIN MATHS

Two layers, no bend, written with column vectors. First layer:

$$\mathbf{h} = \mathbf{W}_1\mathbf{x} = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 3 \\ 1 \end{pmatrix} = \begin{pmatrix} 5 \\ 1 \end{pmatrix}$$

Second layer:

$$\hat{\mathbf{y}} = \mathbf{W}_2\mathbf{h} = \begin{pmatrix} 2 & 0 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 5 \\ 1 \end{pmatrix} = \begin{pmatrix} 10 \\ 6 \end{pmatrix}$$

Now multiply the matrices first:

$$\mathbf{W}_2\mathbf{W}_1 = \begin{pmatrix} 2 & 0 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 2 & 4 \\ 1 & 3 \end{pmatrix}$$

$$(\mathbf{W}_2\mathbf{W}_1)\mathbf{x} = \begin{pmatrix} 2 & 4 \\ 1 & 3 \end{pmatrix} \begin{pmatrix} 3 \\ 1 \end{pmatrix} = \begin{pmatrix} 10 \\ 6 \end{pmatrix}$$

Same answer. One layer with matrix $\begin{pmatrix} 2 & 4 \\ 1 & 3 \end{pmatrix}$ does the whole job.

→

MATRIX FORM

With L linear layers and biases:

$$\hat{\mathbf{y}} = \mathbf{W}_L(\cdots(\mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2)\cdots) + \mathbf{b}_L$$

Multiply everything out. All the $\mathbf{W}$'s collapse into one matrix $\mathbf{W} = \mathbf{W}_L \cdots \mathbf{W}_1$, and all the bias terms collapse into one vector $\mathbf{b}$:

$$\hat{\mathbf{y}} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

That is Book 3's linear model again, whatever L is. Now put a non-linear σ after each layer:

$$\hat{\mathbf{y}} = \sigma(\mathbf{W}_2 \sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2)$$

The σ in the middle stops the two matrices from merging. Depth now buys something.

The three common activation functions

Your slide shows three curves side by side. Here they are with their slopes underneath, because the backward pass (Chapter 4) needs the slope at every neuron.

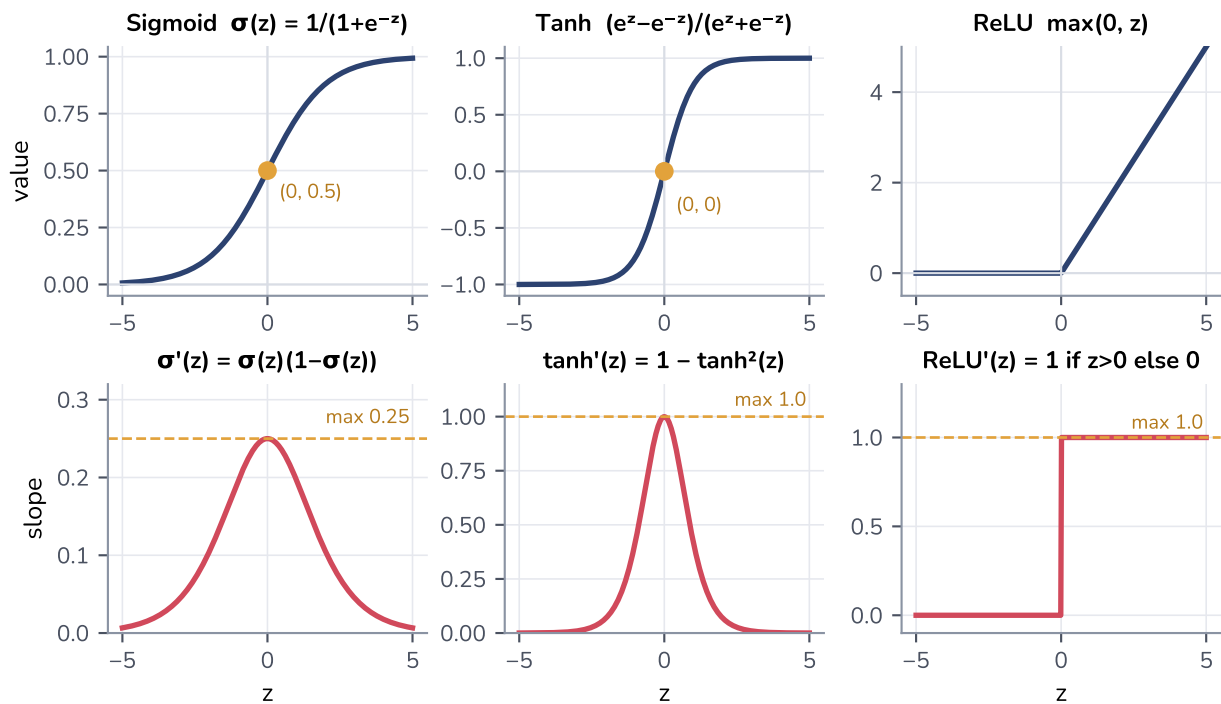


Figure 2.2 Top row: the three activations. Bottom row: their slopes (derivatives). Notice three things. Sigmoid's slope peaks at only 0.25 and dies away on both sides. Tanh's slope peaks at 1, four times larger, but also dies away. ReLU's slope is exactly 1 for every positive input, and 0 for negatives: no dying away on the right.

Property	Sigmoid	Tanh	ReLU	Leaky ReLU
Formula	$\sigma(z) = \frac{1}{1 + e^{-z}}$	$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$\max(0, z)$	$z \text{ if } z > 0, \text{ else } 0.01z$
Range	$(0, 1)$	$(-1, 1)$	$[0, \infty)$	$(-\infty, \infty)$
Zero-centred?	no	yes	no	yes
Differentiable?	yes	yes	not at 0	not at 0
Vanishing gradient	high	moderate	no	no
Computational cost	high (exponential)	high	low	low
Typical use	output layer	hidden layer	hidden layer	hidden layer

Three words in the table: **zero-centred** means outputs can be negative as well as positive, so the next layer gets inputs spread around zero, which helps learning. **Vanishing gradient** means the slope becomes tiny for large $|z|$, so little error signal gets through (Chapter 6 measures this). **Cost** is computer work: an exponential is far slower than a comparison with zero.

WHY RELU WORKS SO WELL

Your slide gives three reasons, and the modern preference is ReLU and its variants for hidden layers.

1. **Computational efficiency.** $\max(0, z)$ is one comparison. No exponentials. Much faster than sigmoid or tanh.
2. **Avoids vanishing gradients.** The slope is 1 whenever $z > 0$, so an error signal passes through unchanged. This is what makes very deep networks trainable.
3. **Sparse activation.** Many neurons output exactly 0. Only some "fire" for a given input, which gives cleaner representations and is closer to how biological neurons behave.

! CAREFUL: THE DYING RELU

If a ReLU neuron's z is negative for every input, it outputs 0 every time, and its slope is 0 every time. In Chapter 4 you will see that a zero slope means a zero gradient, so the neuron never gets updated. It is stuck: a **dead ReLU**. You will meet one in Chapter 5 (hidden unit 2). The fixes on your slide give the negative side a small slope, so the neuron can recover. **Leaky ReLU** uses a fixed small slope like 0.01 or 0.1. **PReLU** learns the slope. **ELU** uses a smooth curve, $e^z - 1$, on the negative side.

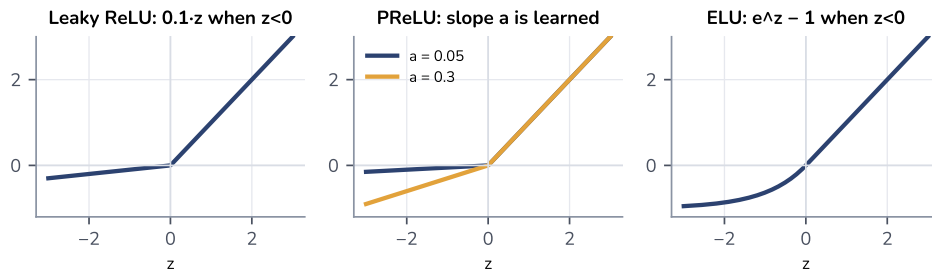


Figure 2.3 The three fixes. All agree with ReLU for $z > 0$; on the left they are not flat, so a stuck neuron can climb out.

Sigmoid, close up

The Numerical deck spends three slides on sigmoid. Its properties: output range $(0, 1)$; smooth and differentiable everywhere; $\sigma(0) = 0.5$; symmetry $\sigma(-z) = 1 - \sigma(z)$; as $z \rightarrow \infty$ it goes to 1, as $z \rightarrow -\infty$ it goes to 0. Key values to remember: $\sigma(-2) \approx 0.12$, $\sigma(0) = 0.5$, $\sigma(2) \approx 0.88$. It is used for binary outputs and for the "gates" inside LSTMs (a later module).

Σ MATHS, STEP BY STEP: THE SLOPE OF SIGMOID

PLAIN MATHS

Step 1. Write sigmoid as a power and use the chain rule (the derivative of e^{-z} is $-e^{-z}$):

$$\sigma(z) = (1 + e^{-z})^{-1} \Rightarrow \frac{d\sigma}{dz} = -(1 + e^{-z})^{-2} \cdot (-e^{-z}) = \frac{e^{-z}}{(1 + e^{-z})^2}$$

Step 2. Split the fraction into two factors:

$$\frac{e^{-z}}{(1 + e^{-z})^2} = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}}$$

Step 3. Recognise the pieces. The first factor is $\sigma(z)$. The second is $1 - \sigma(z)$, because $1 - \frac{1}{1 + e^{-z}} = \frac{e^{-z}}{1 + e^{-z}}$. So:

$$\sigma'(z) = \sigma(z) (1 - \sigma(z))$$

You never need z again: the slope comes from the output alone. Its maximum is at $z = 0$: $0.5 \times 0.5 = 0.25$.

MATRIX FORM

In a network the activation is applied to every entry of a matrix $\mathbf{Z}$ separately (element-wise). The slope is also taken entry by entry:

$$\sigma'(\mathbf{Z}) = \sigma(\mathbf{Z}) \odot (1 - \sigma(\mathbf{Z}))$$

$\odot$ means "multiply matching entries" (the Hadamard product). If $\mathbf{Z}$ is $B \times n$, so is $\sigma'(\mathbf{Z})$. Because the forward pass already computed $\sigma(\mathbf{Z})$, the backward pass gets all the slopes for the price of one subtraction and one multiplication. That is why the "derivative from the output alone" property is called beautiful on your slide.

Tanh, close up

Tanh (the hyperbolic tangent) is a stretched sigmoid, $\tanh(z) = 2\sigma(2z) - 1$: range $(-1, 1)$, zero-centred with $\tanh(0) = 0$, odd ($\tanh(-z) = -\tanh(z)$), going to ± 1 as $z \rightarrow \pm\infty$. Key values: $\tanh(\pm 1) \approx \pm 0.76$. Advantages over sigmoid: zero-centred, stronger gradients (maximum slope 1), faster convergence.

Σ MATHS, STEP BY STEP: THE SLOPE OF TANH (QUOTIENT RULE)

PLAIN MATHS

Write $\tanh = u/v$ with $u = e^z - e^{-z}$ and $v = e^z + e^{-z}$. Then $u' = e^z + e^{-z}$ and $v' = e^z - e^{-z}$ (each one is the other). The quotient rule says $(u/v)' = (u'v - uv')/v^2$:

$$\frac{d \tanh}{dz} = \frac{(e^z + e^{-z})^2 - (e^z - e^{-z})^2}{(e^z + e^{-z})^2}$$

Expand the two squares: $(e^z + e^{-z})^2 = e^{2z} + 2 + e^{-2z}$ and $(e^z - e^{-z})^2 = e^{2z} - 2 + e^{-2z}$.

Their difference is 4:

$$\tanh'(z) = \frac{4}{(e^z + e^{-z})^2}$$

And because $\frac{u^2}{v^2} = \tanh^2$, the same fraction is $1 - \frac{u^2}{v^2}$:

$$\tanh'(z) = 1 - \tanh^2(z)$$

Maximum at $z = 0$: $1 - 0 = 1$.

MATRIX FORM

Element-wise again, from the output alone:

$$\tanh'(\mathbf{Z}) = \mathbf{1} - \tanh(\mathbf{Z}) \odot \tanh(\mathbf{Z})$$

And for ReLU, entry by entry:

$$\text{ReLU}'(\mathbf{Z}) = \begin{cases} 1 & \text{where } \mathbf{Z} > 0 \\ 0 & \text{where } \mathbf{Z} \leq 0 \end{cases}$$

The ReLU slope matrix is just a mask of ones and zeros. Chapter 4 multiplies the error signal by these slope matrices, entry by entry, at every layer.

WORKED EXAMPLE: THE ACTIVATION PROBLEM FROM THE NUMERICAL DECK

Given $\mathbf{z} = (2.0, -1.5, 0.8, -0.3)$, compute each activation and its slope. Here is the full table; below it, one entry of each kind worked by hand.

z	e^{-z}	$\sigma(z)$	$\sigma'(z)$	e^z	$\tanh(z)$	$\tanh'(z)$	ReLU	ReLU'
2.0	0.135	0.881	0.105	7.389	0.964	0.070	2.0	1
-1.5	4.482	0.182	0.149	0.223	-0.905	0.181	0	0
0.8	0.449	0.690	0.214	2.226	0.664	0.559	0.8	1
-0.3	1.350	0.426	0.245	0.741	-0.291	0.915	0	0

1 Sigmoid at $z = 2.0$: $e^{-2.0} = 0.135$, so $\sigma = \frac{1}{1 + 0.135} = \frac{1}{1.135} = 0.881$.
the formula $1/(1 + e^{-z})$

2 Its slope: $\sigma' = 0.881 \times (1 - 0.881) = 0.881 \times 0.119 = 0.105$.
from the output alone

3 Tanh at $z = -1.5$: $e^{-1.5} = 0.223$, $e^{1.5} = 4.482$, so $\tanh = \frac{0.223 - 4.482}{0.223 + 4.482} = \frac{-4.259}{4.705} = -0.905$.
numerator minus, denominator plus

4 Its slope: $1 - (-0.905)^2 = 1 - 0.819 = 0.181$.
squaring removes the sign

- 5 ReLU at $z = 0.8$: $\max(0, 0.8) = 0.8$, slope 1. At $z = -0.3$: $\max(0, -0.3) = 0$, slope 0.
positive passes with slope 1; negative is cut with slope 0

Look down the slope columns of the table. The two negative inputs give sigmoid slopes near their maximum, tanh slopes of 0.181 and 0.915, and ReLU slopes of exactly 0. Which activation you pick decides how much error can flow back through a neuron.

ON YOUR SLIDES

Module 5: slide 10 is the non-linearity definition, slide 11 the three curves with formulas, slide 12 the properties table and "Modern preference: ReLU and variants for hidden layers", slide 13 "Why ReLU works well" and the dying-ReLU caveat. Numerical deck: slides 44–46 sigmoid (properties, three-step derivative, plots), 50–52 tanh (properties, quotient-rule derivative, plots), 53–56 the activation problem. The slide prints $\tanh'(2.0)$ as 0.070; unrounded, $1 - 0.964^2 = 0.0707 \approx 0.071$. Either is fine if you show the working.

TRY IT YOURSELF

Compute $\sigma(1)$, $\sigma'(1)$, $\tanh(0.5)$ and $\tanh'(0.5)$. Use $e^{-1} = 0.368$, $e^{0.5} = 1.649$, $e^{-0.5} = 0.607$. ($\sigma(1) = 1/1.368 = 0.731$; $\sigma' = 0.731 \times 0.269 = 0.197$; $\tanh(0.5) = 1.042/2.256 = 0.462$; $\tanh' = 1 - 0.213 = 0.787$.)

CHECK YOURSELF

1. What happens to a network of five layers if none of them has an activation? **It collapses to one linear layer: $\mathbf{W}_5 \cdots \mathbf{W}_1$ is a single matrix.**
2. What is the largest slope of sigmoid, and where? **0.25, at $z = 0$.**
3. Write $\sigma'(z)$ using only $\sigma(z)$. **$\sigma(z)(1 - \sigma(z))$.**
4. Why is ReLU cheap to compute? **It is one comparison with zero; no exponentials.**
5. What is a dead ReLU and what fixes it? **A neuron whose z is always negative. It outputs 0 and gets zero gradient forever. Leaky ReLU, PReLU or ELU give the negative side a non-zero slope.**

IN ONE BREATH

- Non-linear = not a straight line: $f(ax_1 + bx_2) \neq af(x_1) + bf(x_2)$.
- Without a bend, all layers collapse into one matrix. The bend is what makes depth count.
- Sigmoid: $(0, 1)$, slope $\sigma(1 - \sigma)$, max 0.25. Tanh: $(-1, 1)$, slope $1 - \tanh^2$, max 1. ReLU: $\max(0, z)$, slope 1 or 0.
- ReLU is the default for hidden layers: fast, no vanishing gradient for positives, sparse. Watch for dead units.

The four parts of a deep network

Data, model, loss, learning algorithm — the same four parts as in Book 1. The model is now a stack of layers, so we need a clean way to name every layer, unit and weight.

On your slides: **Module 5, slides 15–26** · DFNN definition · Data representation · Mini-batch · Architecture and notation · Layer-wise computation · Vectorised mini-batch · Total parameters · Loss functions · Mini-batch loss · Mini-batch SGD · Training algorithm (parts 1 and 2)

STORY

Asha is ready to build Nova a real network for the wake word: many inputs, two hidden layers, one output. On the whiteboard she starts writing "the weight from the second unit of the first hidden layer to the third unit of the second hidden layer" and runs out of board. Ravi reads it twice and gives up. "You need names. Short ones." Asha agrees. Before she can train anything, she needs a notation that fits on one line, and she needs to know how many knobs Nova will have.

BIG IDEA

A **deep feed-forward neural network** (DFNN) is a network with one or more hidden layers, where information flows forward through non-linear transformations. It needs the same four parts as every learning system: **data** (input features and targets), a **model** (the layers), an **objective** (a loss to minimise), and a **learning algorithm** (gradient descent, with backpropagation to get the gradients). Every layer does one thing: $\mathbf{z} = \mathbf{h}\mathbf{W} + \mathbf{b}$, then $\mathbf{h} = \sigma(\mathbf{z})$.

THINK OF IT LIKE...

A factory line with L stations. A tray of B half-made items moves from station to station; that tray is a **mini-batch**. Every station has a recipe card (its weights $\mathbf{W}$ and biases $\mathbf{b}$) and a stamp it presses at the end (its activation σ). The tray leaves the last station as B finished items, the predictions. Nobody at station 3 looks at the raw material; they only see what station 2 handed them. That is "feed-forward".

Part 1: data

The dataset is N pairs of (input, target): $D = \{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}_{i=1}^N$. Stack the inputs as rows of one matrix and the targets as rows of another:

$$\mathbf{X} \in \mathbb{R}^{N \times d} = \begin{pmatrix} \mathbf{x}^{(1)\top} \\ \vdots \\ \mathbf{x}^{(N)\top} \end{pmatrix}, \quad \mathbf{Y} \in \mathbb{R}^{N \times K} = \begin{pmatrix} \mathbf{y}^{(1)\top} \\ \vdots \\ \mathbf{y}^{(N)\top} \end{pmatrix}$$

d is the number of input features; K is the output dimension: 1 for regression or a yes/no answer, C for classification with C classes (one-hot rows, Book 4). Training takes a **mini-batch** $\mathcal{B}$ of B rows at a time: $\mathbf{X}_B \in \mathbb{R}^{B \times d}$, $\mathbf{Y}_B \in \mathbb{R}^{B \times K}$, with $B \in \{32, 64, 128, 256\}$. A mini-batch gives a steadier gradient

than one example and is small enough to run in parallel on a GPU.

Part 2: the model, and how to name its parts

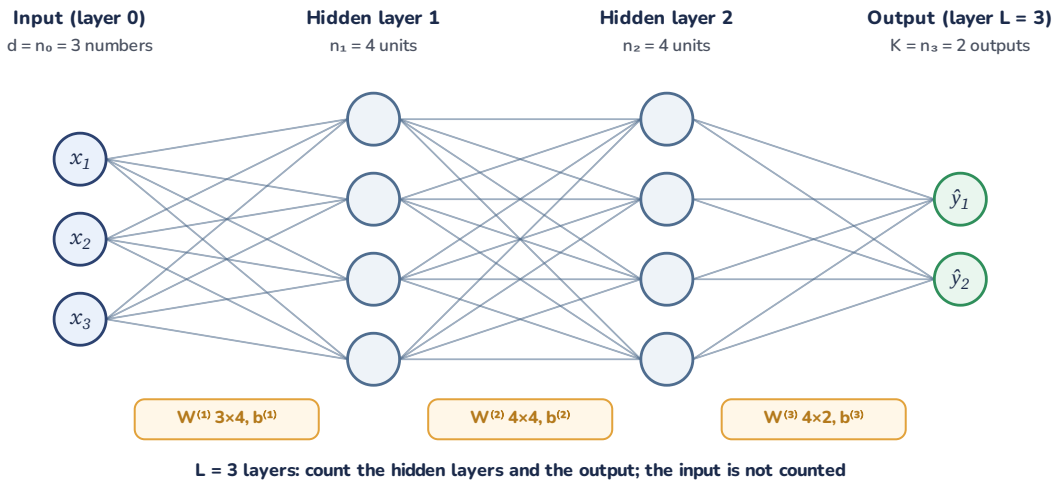


Figure 3.1 The notation on your slide, on a small network. L counts the hidden layers plus the output layer; the input is layer 0 and is not counted. n_ℓ is the number of units in layer ℓ . The input size is $d = n_0$ and the output size is $K = n_L$. Between layer $\ell - 1$ and layer ℓ sits one weight matrix $\mathbf{W}^{(\ell)}$ of shape $n_{\ell-1} \times n_\ell$ and one bias vector $\mathbf{b}^{(\ell)}$ with n_ℓ entries.

Four names to keep. L is the number of layers, counting the output but not the input. n_ℓ is the number of units in layer ℓ , so $n_0 = d$ and $n_L = K$. $\mathbf{W}^{(\ell)}$ and $\mathbf{b}^{(\ell)}$ are the weights and biases of layer ℓ ; the round brackets on top mean "layer number", not a power. $\mathbf{z}^{(\ell)}$ and $\mathbf{h}^{(\ell)}$ are the sums and the activations of layer ℓ , with $\mathbf{h}^{(0)} = \mathbf{x}$ and $\hat{\mathbf{y}} = \mathbf{h}^{(L)}$. Why is the input column not a layer? Because a layer means weights, a bias and an activation, and the input has none of them; it just holds the numbers you feed in.

MATHS, STEP BY STEP: WHAT ONE LAYER COMPUTES

PLAIN MATHS

Look at one unit j in layer ℓ . It reads every activation h_i of the layer before, multiplies each by its own weight w_{ij} , adds them, adds its bias, then applies the bend:

$$z_j^{(\ell)} = \sum_{i=1}^{n_{\ell-1}} h_i^{(\ell-1)} w_{ij}^{(\ell)} + b_j^{(\ell)}$$

$$h_j^{(\ell)} = \sigma^{(\ell)}(z_j^{(\ell)})$$

w_{ij} means "from unit i of the previous layer to unit j of this layer". Start with $\mathbf{h}^{(0)} = \mathbf{x}$ and repeat for $\ell = 1, \dots, L$.

MATRIX FORM (YOUR SLIDE'S ROW CONVENTION)

Write the activations of a layer as a *row* vector. All n_ℓ sums at once:

$$\mathbf{z}^{(\ell)} = \mathbf{h}^{(\ell-1)} \mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)} \in \mathbb{R}^{n_\ell}$$

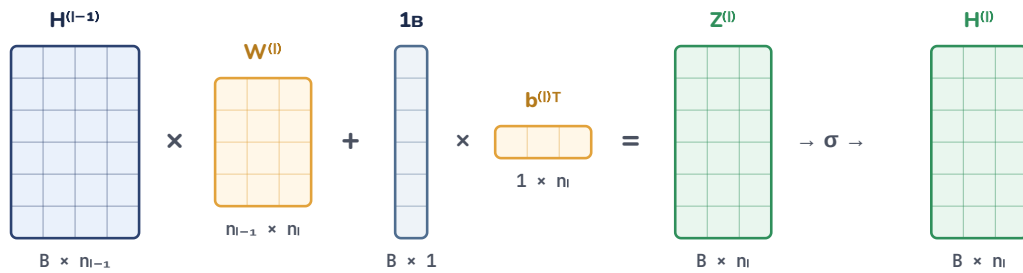
$$\mathbf{h}^{(\ell)} = \sigma^{(\ell)}(\mathbf{z}^{(\ell)}) \in \mathbb{R}^{n_\ell}$$

Shapes: $(1 \times n_{\ell-1})(n_{\ell-1} \times n_\ell) = 1 \times n_\ell$, one number per unit; $\sigma^{(\ell)}$ acts on each entry. For a whole mini-batch, stack the B rows into $\mathbf{H}^{(\ell-1)}$ ($B \times n_{\ell-1}$), with $\mathbf{H}^{(0)} = \mathbf{X}_B$:

$$\mathbf{Z}^{(\ell)} = \mathbf{H}^{(\ell-1)} \mathbf{W}^{(\ell)} + \mathbf{1}_B \mathbf{b}^{(\ell)\top} \in \mathbb{R}^{B \times n_\ell}$$

$$\mathbf{H}^{(\ell)} = \sigma^{(\ell)}(\mathbf{Z}^{(\ell)}), \quad \hat{\mathbf{Y}}_B = \mathbf{H}^{(L)} \in \mathbb{R}^{B \times K}$$

$\mathbf{1}_B$ is a column of B ones, so $\mathbf{1}_B \mathbf{b}^\top$ is the bias row copied B times. One matrix product does $B \times n_\ell$ sums at once: that is why computers train on mini-batches.



One row = one example in the mini-batch. The bias row is copied B times by the column of ones.

Figure 3.2 The shapes in the mini-batch formula. $\mathbf{H}^{(\ell-1)}$ has one row per example and one column per unit of the previous layer. Multiply by $\mathbf{W}^{(\ell)}$ and the columns become the new layer's units. Add the bias row to every row. Notice that B , the number of rows, never changes from layer to layer: the tray keeps its B items.

! CAREFUL: ROWS OR COLUMNS?

Module 5 writes activations as *rows*: $\mathbf{z} = \mathbf{h}\mathbf{W} + \mathbf{b}$, with $\mathbf{W}$ of shape $n_{\ell-1} \times n_{\ell}$. The Numerical deck (and Chapter 5) writes them as *columns*: $\mathbf{z}^{[1]} = \mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]}$, with $\mathbf{W}^{[1]}$ of shape $n_1 \times n_0$ and square brackets for the layer number. These are the same computation written sideways: one matrix is the transpose of the other, $\mathbf{W}_{\text{row}} = \mathbf{W}_{\text{col}}^T$. Whichever you use, weight w from unit i to unit j multiplies h_i and lands in z_j . Figure 3.3 shows both forms on the same numbers.

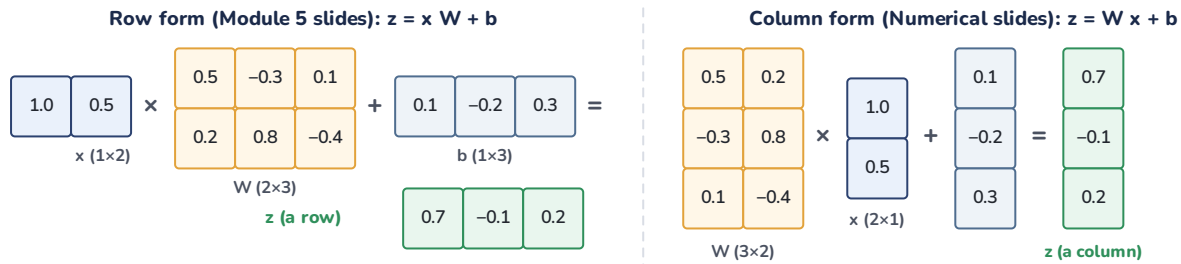


Figure 3.3 The first layer of Chapter 5's network, written both ways. Left: the row form of Module 5. Right: the column form of the Numerical deck. The weight matrices are transposes of each other, and the result $(0.7, -0.1, 0.2)$ is the same, just lying down or standing up.

How many knobs? Counting parameters

Layer ℓ has a weight matrix with $n_{\ell-1} \times n_{\ell}$ entries and a bias vector with n_{ℓ} entries:

$$\text{Parameters}^{(\ell)} = n_{\ell-1} n_{\ell} + n_{\ell} = n_{\ell} (n_{\ell-1} + 1), \quad \text{Total} = \sum_{\ell=1}^L n_{\ell} (n_{\ell-1} + 1)$$

WORKED EXAMPLE: 784 → 256 → 128 → 10

Your slide's network reads a $28 \times 28 = 784$ pixel image of a digit and outputs 10 class scores. Three layers.

- 1 Layer 1: $784 \times 256 + 256 = 200,704 + 256 = 200,960$.
 $n_0 = 784$ inputs into $n_1 = 256$ units, plus 256 biases
- 2 Layer 2: $256 \times 128 + 128 = 32,768 + 128 = 32,896$.
- 3 Layer 3: $128 \times 10 + 10 = 1,280 + 10 = 1,290$.

4 Total: $200,960 + 32,896 + 1,290 = 235,146$ parameters.
a quarter of a million knobs for a small network

Notice that the first layer holds 85% of the parameters: it is the wide input (784) that costs. Deep networks can have millions to billions of parameters, and most sit at the widest layers.

Part 3: the loss

The loss compares one prediction $\hat{\mathbf{y}}$ with its target $\mathbf{y}$. Which loss depends on the task; you met all three in Books 3 and 4.

Regression: MSE

$\ell(\hat{\mathbf{y}}, \mathbf{y}) = \frac{1}{2} \|\hat{\mathbf{y}} - \mathbf{y}\|^2 = \frac{1}{2} \sum_{k=1}^K (\hat{y}_k - y_k)^2$. Output activation: identity.

Binary: BCE ($K = 1$)

$\ell(\hat{y}, y) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$. Output activation: sigmoid, so $\hat{y} \in (0, 1)$.

Multi-class: CCE ($K = C$)

$\ell(\hat{\mathbf{y}}, \mathbf{y}) = -\sum_{k=1}^C y_k \log \hat{y}_k$. $\mathbf{y}$ is one-hot; $\hat{\mathbf{y}}$ comes from softmax.

For a mini-batch, average the per-example losses. This average is the objective J ; it depends on *all* the weights and biases, $\mathbf{W} = \{\mathbf{W}^{(1)}, \dots, \mathbf{W}^{(L)}\}$ and $\mathbf{b} = \{\mathbf{b}^{(1)}, \dots, \mathbf{b}^{(L)}\}$:

$$J(\mathbf{W}, \mathbf{b}) = \frac{1}{B} \sum_{i \in \mathcal{B}} \ell(\hat{\mathbf{y}}^{(i)}, \mathbf{y}^{(i)}), \quad \mathbf{W}^*, \mathbf{b}^* = \arg \min_{\mathbf{W}, \mathbf{b}} J(\mathbf{W}, \mathbf{b})$$

"arg min" means: the settings of all the knobs that make J smallest. That is the goal of training.

Part 4: the learning algorithm

Mini-batch stochastic gradient descent, as in Book 4, but now every layer has its own knobs. For each layer ℓ , step against the gradient:

$$\mathbf{W}^{(\ell)} \leftarrow \mathbf{W}^{(\ell)} - \eta \frac{\partial J}{\partial \mathbf{W}^{(\ell)}}, \quad \mathbf{b}^{(\ell)} \leftarrow \mathbf{b}^{(\ell)} - \eta \frac{\partial J}{\partial \mathbf{b}^{(\ell)}}$$

η (eta) is the learning rate, the step size. $\partial J / \partial \mathbf{W}^{(\ell)}$ is a matrix the same shape as $\mathbf{W}^{(\ell)}$: for every weight, "how much does J change if I nudge this weight". The challenge on your slide: **how do we compute all these gradients efficiently?** The weights of layer 1 affect J only through layers 2, 3, ... The answer is **backpropagation**: the chain rule, applied backward through the network. It is the key idea that made deep learning possible, and it is Chapter 4.

THE WHOLE TRAINING ALGORITHM

Your slides split it into Algorithm 1 (forward pass and loss) and Algorithm 2 (backward pass and update). One pass through all the mini-batches is an **epoch**; then shuffle and go again.

1. **Initialise** every $\mathbf{W}^{(\ell)}$ to small random numbers and every $\mathbf{b}^{(\ell)}$ to zeros.
2. **For each epoch** $t = 1, \dots, T$: shuffle the dataset; then **for each mini-batch**:
 - Forward: $\mathbf{H}^{(0)} \leftarrow \mathbf{X}_B$; for $\ell = 1, \dots, L$: $\mathbf{Z}^{(\ell)} \leftarrow \mathbf{H}^{(\ell-1)} \mathbf{W}^{(\ell)} + \mathbf{1}_B \mathbf{b}^{(\ell)\top}$, $\mathbf{H}^{(\ell)} \leftarrow \sigma^{(\ell)}(\mathbf{Z}^{(\ell)})$; $\hat{\mathbf{Y}}_B \leftarrow \mathbf{H}^{(L)}$.
 - Loss: $J \leftarrow \frac{1}{B} \sum_{i \in \mathcal{B}} \ell(\hat{\mathbf{y}}^{(i)}, \mathbf{y}^{(i)})$.
 - Backward: $\delta^{(L)} \leftarrow \partial J / \partial \mathbf{Z}^{(L)}$; for $\ell = L - 1$ down to 1: $\delta^{(\ell)} \leftarrow (\delta^{(\ell+1)} \mathbf{W}^{(\ell+1)\top}) \odot \sigma'^{(\ell)}(\mathbf{Z}^{(\ell)})$.
 - Gradients: for each ℓ : $\partial J / \partial \mathbf{W}^{(\ell)} \leftarrow \frac{1}{B} \mathbf{H}^{(\ell-1)\top} \delta^{(\ell)}$, $\partial J / \partial \mathbf{b}^{(\ell)} \leftarrow \frac{1}{B} \mathbf{1}_B^\top \delta^{(\ell)}$.
 - Update: for each ℓ : $\mathbf{W}^{(\ell)} \leftarrow \mathbf{W}^{(\ell)} - \eta \partial J / \partial \mathbf{W}^{(\ell)}$, $\mathbf{b}^{(\ell)} \leftarrow \mathbf{b}^{(\ell)} - \eta \partial J / \partial \mathbf{b}^{(\ell)}$.

The δ lines are explained in Chapter 4; the next chapter but one runs the whole loop once by hand.

📖 ON YOUR SLIDES

Slide 15: definition and four components ("same framework as linear models, but with added depth and non-linearity"). Slide 16: $\mathbf{X} \in \mathbb{R}^{N \times d}$, $\mathbf{Y} \in \mathbb{R}^{N \times K}$. Slide 17: mini-batch and batch sizes. Slide 18: the architecture picture and notation box. Slide 19: layer-wise computation, equations (10)–(15). Slide 20: vectorised mini-batch, (16)–(19). Slide 21: parameter count, (20)–(21), 235,146. Slides 22–24: losses, mini-batch loss and arg min, SGD update and "Solution: Backpropagation algorithm!". Slides 25–26: Algorithms 1 and 2.

✅ CHECK YOURSELF

1. A network has two hidden layers and an output layer. What is L ? **3. The input is not counted.**
2. What is the shape of $\mathbf{W}^{(\ell)}$ in the row convention? $n_{\ell-1} \times n_{\ell}$: **previous units by current units.**
3. What does $\mathbf{1}_B \mathbf{b}^{(\ell)\top}$ do? **Copies the bias row B times so every example in the batch gets the same bias.**
4. How many parameters in a 2-3-2 network (the one you will train by hand)? $3 \times (2 + 1) + 2 \times (3 + 1) = 9 + 8 = 17$.
5. Which loss goes with which output activation? **MSE with identity, BCE with sigmoid, CCE with softmax.**

🧠 IN ONE BREATH

- Data $\mathbf{X}$ ($N \times d$), targets $\mathbf{Y}$ ($N \times K$); train on mini-batches of B rows.
- L layers (output counted, input not); layer ℓ has n_{ℓ} units, $\mathbf{W}^{(\ell)}$ is $n_{\ell-1} \times n_{\ell}$, $\mathbf{b}^{(\ell)}$ has n_{ℓ} entries.
- Every layer: $\mathbf{Z} = \mathbf{H}\mathbf{W} + \mathbf{1}_B \mathbf{b}^{\top}$, $\mathbf{H} = \sigma(\mathbf{Z})$. Row form and column form are the same thing sideways.
- Parameters per layer: $n_{\ell}(n_{\ell-1} + 1)$. Loss: MSE / BCE / CCE, averaged over the batch. Update every layer with SGD; backprop supplies the gradients.

Forward, loss, backward

The forward pass makes predictions and keeps notes. The loss scores them. The backward pass sends one error signal back through the layers and turns it into every gradient at once.

On your slides: **Module 5, slides 28–39** · Forward pass and algorithm · Vectorised forward pass and complexity · Loss computation · Output layer gradient · Backpropagation · Backpropagating error signals · Parameter gradients · Backprop algorithm · Activation derivatives · Efficiency · Parameter update

STORY

Asha has the training loop on the board, but one line is a mystery: "compute the gradient for every layer". She knows how to nudge one weight, rerun Nova, and see if the loss drops. Nova has 235,146 weights. Ravi does the sum: "If one forward pass takes a millisecond, that is four minutes for one step. You want a million steps." Asha needs a way to find *all* the gradients from *one* pass. That way exists. It sends the error backwards.

BIG IDEA

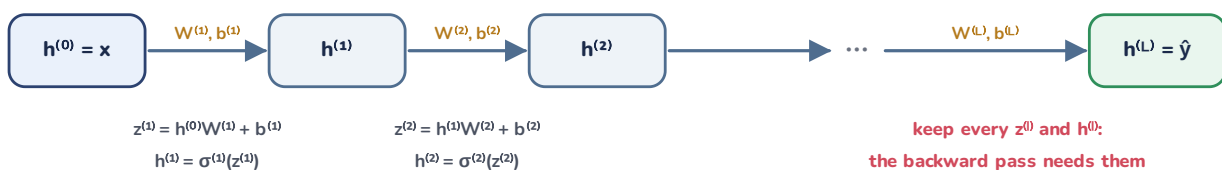
Training a layer needs to know how much the loss changes when that layer's sums $\mathbf{Z}^{(\ell)}$ change. Call that the **error signal** $\delta^{(\ell)}$. At the output it is simply $\frac{1}{B}(\hat{\mathbf{Y}} - \mathbf{Y})$. Each earlier layer gets its signal from the one after it: multiply by the next weights transposed, then by the slope of the activation. From $\delta^{(\ell)}$ the two gradients of layer ℓ follow in one line each. One forward pass plus one backward pass gives every gradient in the network. That is **backpropagation**.

THINK OF IT LIKE...

A customer sends back a dish. The complaint reaches the last cook (the output layer), who passes blame back to the cooks before, in proportion to how much of each one's work went into the dish (the weights, transposed). A cook who was idle for this dish (slope zero) gets no blame. Each cook fixes their own recipe by their share of the blame times what they were given to work with (their input). Nobody remakes the dish 235,146 times.

The forward pass

Given an input $\mathbf{x} \in \mathbb{R}^d$, the forward pass computes the prediction $\hat{\mathbf{y}} \in \mathbb{R}^K$ one layer at a time, using the row form from Chapter 3.



Forward pass: each layer reads the one before it. Cost $\approx B \times \sum_{l=1}^L n_l n_{l-1}$ multiplications.

Figure 4.1 The forward chain on your slide: each arrow multiplies by $\mathbf{W}^{(\ell)}$, adds $\mathbf{b}^{(\ell)}$, applies $\sigma^{(\ell)}$. Notice the red note: every $\mathbf{z}$ and $\mathbf{h}$ is stored for the backward pass.

Your slide's Algorithm 3: set $\mathbf{h}^{(0)} \leftarrow \mathbf{x}$; for $\ell = 1, \dots, L$ compute $\mathbf{z}^{(\ell)} \leftarrow \mathbf{h}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)}$ and $\mathbf{h}^{(\ell)} \leftarrow \sigma^{(\ell)}(\mathbf{z}^{(\ell)})$, storing both; output $\hat{\mathbf{y}} \leftarrow \mathbf{h}^{(L)}$; return $\hat{\mathbf{y}}$ and all the stored $\{\mathbf{h}^{(\ell)}, \mathbf{z}^{(\ell)}\}$. For a mini-batch, use the matrices of Chapter 3: $\mathbf{H}^{(0)} = \mathbf{X}_B$, $\mathbf{Z}^{(\ell)} = \mathbf{H}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{1}_B\mathbf{b}^{(\ell)\top}$, $\mathbf{H}^{(\ell)} = \sigma^{(\ell)}(\mathbf{Z}^{(\ell)})$, $\hat{\mathbf{Y}}_B = \mathbf{H}^{(L)}$. Layer ℓ needs $B \times n_{\ell-1} \times n_\ell$ multiplications, so the pass costs about

$$O\left(B \sum_{\ell=1}^L n_{\ell-1} n_\ell\right)$$

For the $784 \rightarrow 256 \rightarrow 128 \rightarrow 10$ network with $B = 128$: $128 \times (784 \cdot 256 + 256 \cdot 128 + 128 \cdot 10) = 128 \times 234,752 \approx 30$ million multiplications per mini-batch. A GPU does that in well under a millisecond.

The loss. After the forward pass, score each example with the loss for the task (MSE, BCE or CCE) and average: $J = \frac{1}{B} \sum_{i=1}^B \ell^{(i)}$. One number: how wrong the mini-batch is. Everything that follows is about making it smaller.

The first error signal: the output layer

Backpropagation starts at the output. Define the output error signal as the gradient of J with respect to the output layer's *pre-activations* $\mathbf{Z}^{(L)}$ (the sums, before the last activation):

$$\delta^{(L)} = \frac{\partial J}{\partial \mathbf{Z}^{(L)}} \in \mathbb{R}^{B \times K}$$

Your slide makes a remarkable claim: for the three standard pairings, **MSE + identity**, **BCE + sigmoid**, and **CCE + softmax**, this gradient is the same:

$$\delta^{(L)} = \frac{1}{B} (\hat{\mathbf{Y}} - \mathbf{Y})$$

Prediction minus target, divided by the batch size. Why the same? Because each loss was designed for its activation: the slope of the activation exactly cancels the awkward part of the slope of the loss. Here is the sigmoid case in full; the softmax case is derived in Chapter 8.

Σ MATHS, STEP BY STEP: WHY $\Delta^{(L)} = \hat{\mathbf{Y}} - \mathbf{Y}$

PLAIN MATHS (ONE EXAMPLE, BCE + SIGMOID)

The chain: $z \rightarrow \hat{y} = \sigma(z) \rightarrow \ell = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$. By the chain rule, $\frac{\partial \ell}{\partial z} = \frac{\partial \ell}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z}$.

Slope of the loss (Chapter 8 derives it): $\frac{\partial \ell}{\partial \hat{y}} = \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})}$.

Slope of sigmoid (Chapter 2): $\frac{\partial \hat{y}}{\partial z} = \hat{y}(1 - \hat{y})$.

Multiply; the $\hat{y}(1 - \hat{y})$ cancels:

$$\frac{\partial \ell}{\partial z} = \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})} \cdot \hat{y}(1 - \hat{y}) = \hat{y} - y$$

For MSE + identity it is even quicker: $\ell = \frac{1}{2}(\hat{y} - y)^2$ with $\hat{y} = z$, so $\partial \ell / \partial z = \hat{y} - y$ directly.

MATRIX FORM

Stack the B examples. J is the average of the B losses, so each example's gradient is divided by B :

$$\delta^{(L)} = \frac{\partial J}{\partial \mathbf{Z}^{(L)}} = \frac{1}{B} (\hat{\mathbf{Y}} - \mathbf{Y}) \in \mathbb{R}^{B \times K}$$

Row i is example i 's "prediction minus target". This matrix is the seed of everything: every other gradient in the network is computed from it.

Tiny check, $B = 2$, one output: $\hat{\mathbf{Y}} = (0.8, 0.3)^\top$, $\mathbf{Y} = (1, 0)^\top$ give $\delta^{(L)} = \frac{1}{2}(-0.2, 0.3)^\top = (-0.1, 0.15)^\top$: example 1 too low (push up), example 2 too high (push down).

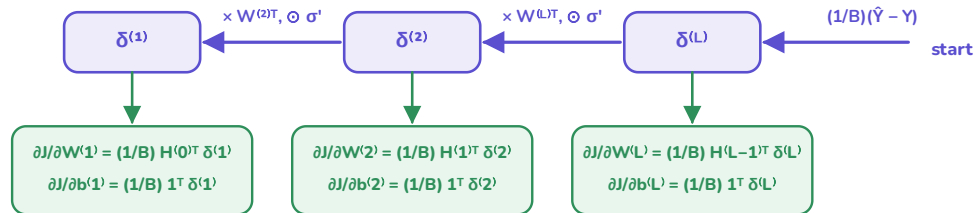
Sending the signal backwards

Define the same thing for every layer: $\delta^{(\ell)} = \partial J / \partial \mathbf{Z}^{(\ell)} \in \mathbb{R}^{B \times n_\ell}$, "how much the loss changes when the sums of layer ℓ change". Two steps: (1) pass the signals back, $\delta^{(L)} \rightarrow \delta^{(L-1)} \rightarrow \dots \rightarrow \delta^{(1)}$; (2) turn each $\delta^{(\ell)}$ into its layer's gradients.

Forward (left to right)



Backward (right to left): error signals δ



Each δ is used twice: passed one layer back, and turned into that layer's two gradients.

Figure 4.2 Top: forward, left to right. Middle: the error signals, right to left, each step multiplying by the next weights transposed and the activation slope. Bottom: each $\delta^{(\ell)}$ gives its layer's two gradients. Notice $\delta^{(\ell)}$ is used twice.

Σ MATHS, STEP BY STEP: THE BACKPROP EQUATION

PLAIN MATHS (ONE UNIT)

Take unit j of layer ℓ . Its sum z_j affects the loss only through its output $h_j = \sigma(z_j)$, and h_j affects the loss only through the sums of the next layer, $z_k^{(\ell+1)} = \sum_j h_j w_{jk}^{(\ell+1)} + b_k$.

Chain rule, one link at a time:

$$\delta_j^{(\ell)} = \frac{\partial J}{\partial z_j} = \frac{\partial J}{\partial h_j} \cdot \frac{\partial h_j}{\partial z_j}$$

The second factor is the activation slope, $\sigma'(z_j)$. The first factor adds up the blame from every unit k of the next layer that used h_j , weighted by the weight it used:

$$\frac{\partial J}{\partial h_j} = \sum_{k=1}^{n_{\ell+1}} \delta_k^{(\ell+1)} w_{jk}^{(\ell+1)}$$

Put together:

$$\delta_j^{(\ell)} = \left(\sum_k \delta_k^{(\ell+1)} w_{jk}^{(\ell+1)} \right) \sigma'(z_j^{(\ell)})$$

Blame in, times weight, times slope.

MATRIX FORM (YOUR SLIDE'S EQUATIONS 40–43)

Chain rule for the whole layer:

$$\delta^{(\ell)} = \frac{\partial J}{\partial \mathbf{Z}^{(\ell)}} = \frac{\partial J}{\partial \mathbf{H}^{(\ell)}} \cdot \frac{\partial \mathbf{H}^{(\ell)}}{\partial \mathbf{Z}^{(\ell)}}$$

First factor: $\mathbf{Z}^{(\ell+1)} = \mathbf{H}^{(\ell)} \mathbf{W}^{(\ell+1)} + \dots$, so the blame comes back through $\mathbf{W}^{(\ell+1)}$ transposed:

$$\frac{\partial J}{\partial \mathbf{H}^{(\ell)}} = \delta^{(\ell+1)} \mathbf{W}^{(\ell+1)\top}$$

Shapes: $(B \times n_{\ell+1})(n_{\ell+1} \times n_\ell) = B \times n_\ell$.

Second factor: the activation is element-wise, so its slope is the element-wise matrix $\sigma'(\mathbf{Z}^{(\ell)})$. Multiply entry by entry:

$$\delta^{(\ell)} = (\delta^{(\ell+1)} \mathbf{W}^{(\ell+1)\top}) \odot \sigma'(\mathbf{Z}^{(\ell)})$$

$\odot$ is the Hadamard (entry-by-entry) product.

Repeat from $\ell = L - 1$ down to 1.

From error signals to gradients

Layer ℓ computed $\mathbf{Z}^{(\ell)} = \mathbf{H}^{(\ell-1)} \mathbf{W}^{(\ell)} + \mathbf{1}_B \mathbf{b}^{(\ell)\top}$. The gradient with respect to a weight is "error signal at the unit it feeds, times the input it multiplied", summed over the batch. With matrices that sum is a transpose product:

$$\frac{\partial J}{\partial \mathbf{W}^{(\ell)}} = \frac{\partial J}{\partial \mathbf{Z}^{(\ell)}} \frac{\partial \mathbf{Z}^{(\ell)}}{\partial \mathbf{W}^{(\ell)}} = \frac{1}{B} \mathbf{H}^{(\ell-1)\top} \delta^{(\ell)} \in \mathbb{R}^{n_{\ell-1} \times n_{\ell}}, \quad \frac{\partial J}{\partial \mathbf{b}^{(\ell)}} = \frac{1}{B} \mathbf{1}_B^\top \delta^{(\ell)} \in \mathbb{R}^{n_{\ell}}$$

Check the shapes: $(n_{\ell-1} \times B)(B \times n_{\ell}) = n_{\ell-1} \times n_{\ell}$, the same shape as $\mathbf{W}^{(\ell)}$, as it must be: one gradient entry per weight. The bias gradient adds the B rows of $\delta^{(\ell)}$ into one row of n_{ℓ} numbers, because every example's bias is the same bias. Your slide's key line: **the error signal $\delta^{(\ell)}$ is shared by both gradients.**

! CAREFUL: COUNT THE $\frac{1}{B}$ ONCE

Your slides put a $\frac{1}{B}$ in $\delta^{(L)} = \frac{1}{B}(\hat{\mathbf{Y}} - \mathbf{Y})$ and again in $\frac{\partial J}{\partial \mathbf{W}} = \frac{1}{B} \mathbf{H}^\top \delta$. The batch average happens once, so use one of the two: if $\delta^{(L)}$ already carries the $\frac{1}{B}$, the gradients are simply $\mathbf{H}^\top \delta$ and $\mathbf{1}^\top \delta$; if you define $\delta^{(L)} = \hat{\mathbf{Y}} - \mathbf{Y}$, put the $\frac{1}{B}$ in the gradient formulas. Both give the same numbers (in the tiny check, $\mathbf{H}^{(L-1)} = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}$ gives $\mathbf{H}^\top \delta^{(L)} = (-0.1, -0.05)^\top$, which a finite-difference check confirms). Counting it twice divides every gradient by B again; the learning rate would absorb it, but be consistent.

THE BACKPROPAGATION ALGORITHM (YOUR SLIDE'S ALGORITHM 4)

1. Compute the output error signal $\delta^{(L)} \leftarrow \partial J / \partial \mathbf{Z}^{(L)}$, which depends on the loss: for the standard pairings, $\frac{1}{B}(\hat{\mathbf{Y}} - \mathbf{Y})$.
2. For $\ell = L - 1$ down to 1: $\delta^{(\ell)} \leftarrow (\delta^{(\ell+1)} \mathbf{W}^{(\ell+1)\top}) \odot \sigma'(\mathbf{Z}^{(\ell)})$.
3. For $\ell = 1, \dots, L$: $\partial J / \partial \mathbf{W}^{(\ell)} \leftarrow \frac{1}{B} \mathbf{H}^{(\ell-1)\top} \delta^{(\ell)}$ and $\partial J / \partial \mathbf{b}^{(\ell)} \leftarrow \frac{1}{B} \mathbf{1}_B^\top \delta^{(\ell)}$.
4. Return all the gradients.

Step 2 needs the activation slopes at the stored $\mathbf{Z}^{(\ell)}$ (Chapter 2): sigmoid $\sigma(z)(1 - \sigma(z))$, tanh $1 - \tanh^2(z)$, ReLU 1 if $z > 0$ else 0. All three come from numbers the forward pass already stored.

Why backpropagation is fast

The naive way nudges each of the P parameters in turn and reruns the forward pass: P forward passes for one gradient, cost $O(P \times C_{\text{forward}})$. Backpropagation reuses the stored $\mathbf{Z}$ and $\mathbf{H}$: one forward pass plus one backward pass of about the same cost, $O(C_{\text{forward}} + C_{\text{backward}}) \approx 2 \times C_{\text{forward}}$. Speed-up about $P/2$: for $P = 200,000$, roughly 100,000 times faster. Your slide calls it *dynamic programming via the chain rule*: each $\delta^{(\ell)}$ is computed once and reused by every parameter of its layer and every layer before it.

The update

With every gradient in hand, take the step. For a single example (your slide's vector form), with $\mathbf{h}^{(\ell-1)}$ a row and $\delta^{(\ell)}$ a row:

$$\mathbf{W}^{(\ell)} \leftarrow \mathbf{W}^{(\ell)} - \eta \mathbf{h}^{(\ell-1)\top} \delta^{(\ell)}, \quad \mathbf{b}^{(\ell)} \leftarrow \mathbf{b}^{(\ell)} - \eta \delta^{(\ell)}$$

$\mathbf{h}^{(\ell-1)\top} \delta^{(\ell)}$ is $n_{\ell-1} \times n_{\ell}$: entry (i, j) is "input i times error at unit j ", the gradient of w_{ij} . The minus sign moves every parameter the way that reduces the loss. For a mini-batch use the averaged gradients. Then the next mini-batch.

ON YOUR SLIDES

Slide 28: forward chain; 29: Algorithm 3 ("Store activations for backpropagation"); 30: vectorised forward pass, complexity (34); 31: loss; 32: $\delta^{(L)} = \frac{1}{B}(\hat{\mathbf{Y}} - \mathbf{Y})$ for the three pairings; 33: $\delta^{(\ell)}$ and the two steps; 34: chain-rule derivation (40)–(43) with $\odot$; 35: parameter gradients (44)–(45); 36: Algorithm 4; 37: activation derivatives (46)–(48); 38: efficiency, speed-up $P/2$; 39: the update (49)–(50).

CHECK YOURSELF

1. What must the forward pass store for the backward pass? **Every $\mathbf{Z}^{(\ell)}$ (for the slopes) and every $\mathbf{H}^{(\ell)}$ (as the inputs of the next layer).**
2. What is $\delta^{(L)}$ for CCE + softmax? **$\frac{1}{B}(\hat{\mathbf{Y}} - \mathbf{Y})$, the same as for the other two pairings.**
3. Write the backprop equation. **$\delta^{(\ell)} = (\delta^{(\ell+1)} \mathbf{W}^{(\ell+1)\top}) \odot \sigma'(\mathbf{Z}^{(\ell)})$.**
4. What is the shape of $\partial J / \partial \mathbf{W}^{(\ell)}$? **$n_{\ell-1} \times n_{\ell}$, the same as $\mathbf{W}^{(\ell)}$.**
5. Why is backprop about $P/2$ times faster than the naive method? **Naive: P forward passes. Backprop: one forward and one backward, about two forward passes.**

IN ONE BREATH

- Forward: $\mathbf{Z} = \mathbf{H}\mathbf{W} + \mathbf{1}\mathbf{b}^\top$, $\mathbf{H} = \sigma(\mathbf{Z})$, store everything; cost $O(B \sum n_{\ell-1} n_{\ell})$.
- Output signal: $\delta^{(L)} = \frac{1}{B}(\hat{\mathbf{Y}} - \mathbf{Y})$ for all three loss-activation pairs.
- Back one layer: $\delta^{(\ell)} = (\delta^{(\ell+1)} \mathbf{W}^{(\ell+1)\top}) \odot \sigma'(\mathbf{Z}^{(\ell)})$: weights transposed, times slope.
- Gradients: $\frac{1}{B} \mathbf{H}^{(\ell-1)\top} \delta^{(\ell)}$ and $\frac{1}{B} \mathbf{1}^\top \delta^{(\ell)}$. Update: $\mathbf{W} \leftarrow \mathbf{W} - \eta \partial J / \partial \mathbf{W}$.
- One forward + one backward = every gradient. About $P/2$ times faster than nudging each parameter.

By hand: one whole training step

A 2-3-2 network, one input, one target. Forward, loss, backward, update — every number worked out on the whiteboard, with a picture of the network at every step.

On your slides: **Numerical deck, slides 3–31** · Forward propagation (steps 0–4) · Computational graph: forward · Backward propagation (loss, output error, output gradients, hidden error, hidden gradients) · Computational graph: backward · Weight update · Computational graph: update

STORY

Asha draws the smallest network that can show everything: two inputs, three hidden units with ReLU, two outputs with sigmoid. Output 1 answers "is it my name?" and output 2 answers "is it a question?". She plays one clip of herself saying "Nova." — her name, not a question — so the target is $y = (1, 0)$. Nova's screen shows two features of the clip: $x = (1.0, 0.5)$. Ravi sits down with a calculator. "Every number. No skipping." Asha uncaps the marker.

BIG IDEA

One training step has four parts, and each part is just Chapter 4 with numbers in it. **Forward:** multiply, add bias, bend, twice. **Loss:** compare the two outputs with y . **Backward:** an error signal at the outputs, gradients for layer 2, the signal passed back through $W^{[2]}$ and the ReLU slopes, gradients for layer 1. **Update:** every one of the 17 parameters moves a little against its gradient. After the step the loss is smaller. That is learning.

THINK OF IT LIKE...

Tuning a guitar string by ear. Pluck it (forward pass). Hear how far it is from the right note (loss). Work out which way the peg must turn, and how much (backward pass). Turn it a little, not all the way (update, small learning rate). Pluck again: closer. Training repeats this thousands of times; here we do it once, slowly.

The problem, and the deck's notation

The Numerical deck writes the layer number in **square brackets**, $W^{[1]}$, and uses **column vectors**: $z^{[1]} = W^{[1]}x + b^{[1]}$. So $W^{[1]}$ has one *row* per hidden unit and one *column* per input (3×2), the transpose of Module 5's row convention (Chapter 3, Figure 3.3). The activations are called a instead of h , and the learning rate α instead of η . Same maths, different letters.

$$x = \begin{pmatrix} 1.0 \\ 0.5 \end{pmatrix}, \quad W^{[1]} = \begin{pmatrix} 0.5 & 0.2 \\ -0.3 & 0.8 \\ 0.1 & -0.4 \end{pmatrix}, \quad b^{[1]} = \begin{pmatrix} 0.1 \\ -0.2 \\ 0.3 \end{pmatrix}, \quad y = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

$$W^{[2]} = \begin{pmatrix} 0.4 & -0.1 & 0.6 \\ 0.2 & 0.7 & -0.3 \end{pmatrix}, \quad b^{[2]} = \begin{pmatrix} 0.2 \\ -0.1 \end{pmatrix}$$

Hidden activation: ReLU. Output activation: sigmoid. Loss: binary cross-entropy on each output. Read $W^{[1]}$ row by row: row 1 is hidden unit 1's two weights (0.5, 0.2), row 2 is unit 2's (−0.3, 0.8), row 3 is unit 3's (0.1, −0.4).

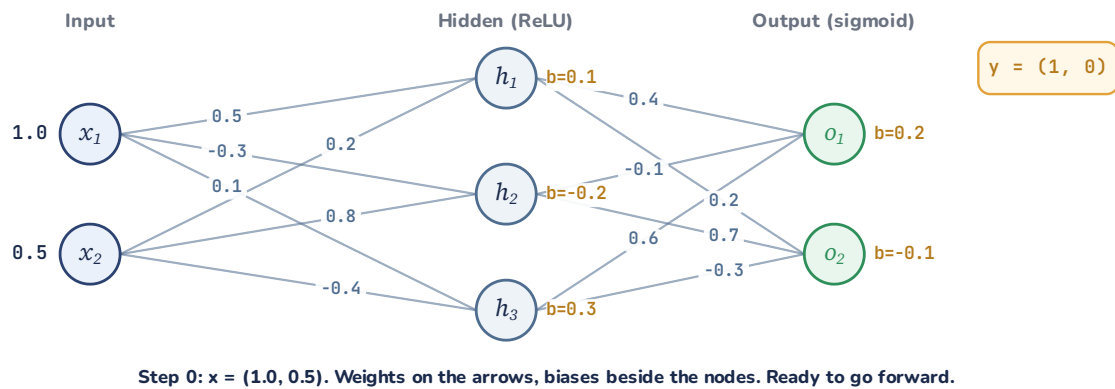


Figure 5.1 Step 0: the network with every weight on its arrow and every bias beside its unit. The number on the arrow from x_i to h_j is entry (j, i) of $W^{[1]}$. Check one: $x_2 \rightarrow h_3$ carries -0.4 , the entry in row 3, column 2.

Forward pass

STEP 1: THE HIDDEN SUMS $Z^{[1]} = W^{[1]}X + B^{[1]}$

- 1 Unit 1: $0.5 \times 1.0 + 0.2 \times 0.5 = 0.5 + 0.1 = 0.6$; add bias 0.1 : $z_1^{[1]} = 0.7$.
row 1 of $W^{[1]}$ times x , plus $b_1^{[1]}$
- 2 Unit 2: $-0.3 \times 1.0 + 0.8 \times 0.5 = -0.3 + 0.4 = 0.1$; add bias -0.2 : $z_2^{[1]} = -0.1$.
- 3 Unit 3: $0.1 \times 1.0 + (-0.4) \times 0.5 = 0.1 - 0.2 = -0.1$; add bias 0.3 : $z_3^{[1]} = 0.2$.

$$z^{[1]} = \begin{pmatrix} 0.6 \\ 0.1 \\ -0.1 \end{pmatrix} + \begin{pmatrix} 0.1 \\ -0.2 \\ 0.3 \end{pmatrix} = \begin{pmatrix} 0.7 \\ -0.1 \\ 0.2 \end{pmatrix}$$

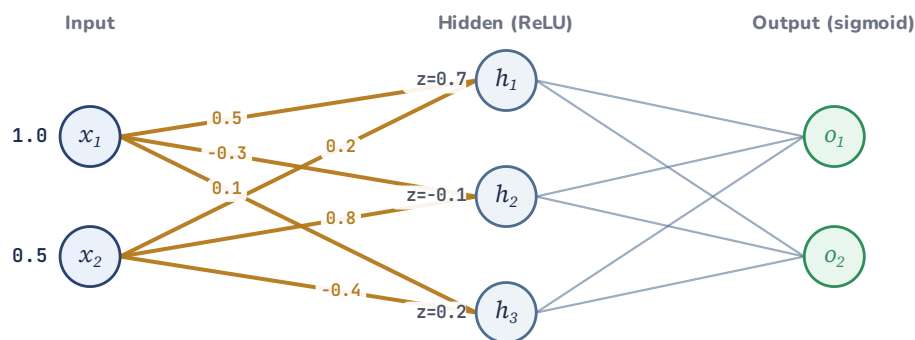
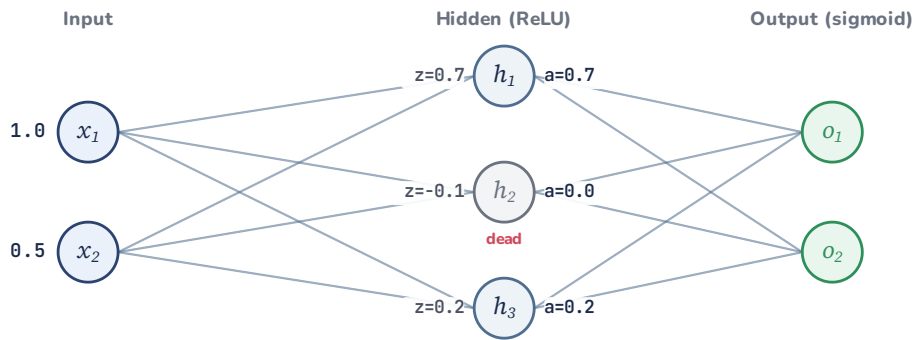


Figure 5.2 Step 1: the six arrows into the hidden layer are live; each unit's z is on its left. Unit 2's sum is negative.

STEP 2: THE HIDDEN ACTIVATIONS $A^{[1]} = \text{RELU}(Z^{[1]})$

- 1 $a_1^{[1]} = \max(0, 0.7) = 0.7$.
- 2 $a_2^{[1]} = \max(0, -0.1) = 0.0$.
negative in, zero out: this unit is dead for this input
- 3 $a_3^{[1]} = \max(0, 0.2) = 0.2$.

So $a^{[1]} = (0.7, 0.0, 0.2)^\top$.



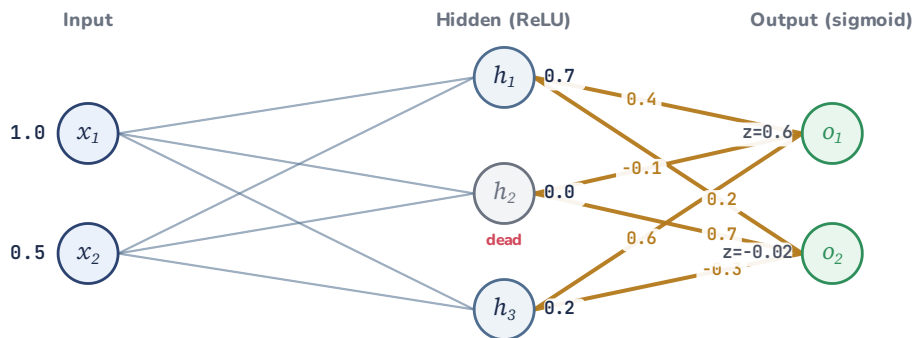
Step 2: ReLU keeps positives, zeroes negatives $\rightarrow a^{(1)} = (0.7, 0.0, 0.2)$. Unit 2 is dead.

Figure 5.3 Step 2: z on the left, a on the right. Unit 2 is grey: it outputs 0, contributes nothing to the next layer, and will receive no blame.

STEP 3: THE OUTPUT SUMS $Z^{[2]} = W^{[2]} A^{[1]} + B^{[2]}$

- 1 Output 1: $0.4 \times 0.7 + (-0.1) \times 0.0 + 0.6 \times 0.2 = 0.28 + 0 + 0.12 = 0.4$; add bias 0.2: $z_1^{[2]} = 0.6$.
the dead unit's weight -0.1 is multiplied by 0
- 2 Output 2: $0.2 \times 0.7 + 0.7 \times 0.0 + (-0.3) \times 0.2 = 0.14 + 0 - 0.06 = 0.08$; add bias -0.1 : $z_2^{[2]} = -0.02$.

So $z^{[2]} = (0.4, 0.08)^\top + (0.2, -0.1)^\top = (0.6, -0.02)^\top$.



Step 3: each output unit adds up (weight $\times$ a) + bias $\rightarrow z^{(2)} = (0.6, -0.02)$

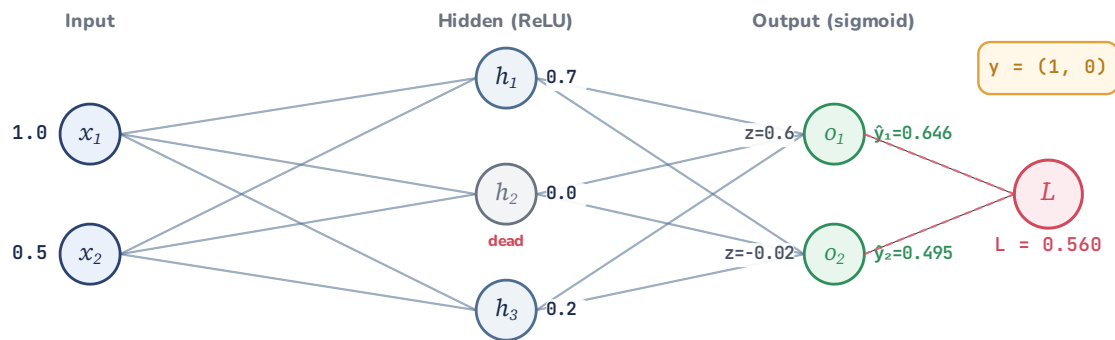
Figure 5.4 Step 3: the arrows into the output layer multiply the hidden activations 0.7, 0.0, 0.2. Each output's z is on its left.

STEP 4: THE OUTPUTS $A^{[2]} = \Sigma(Z^{[2]})$

- 1 $\hat{y}_1 = \sigma(0.6) = \frac{1}{1 + e^{-0.6}} = \frac{1}{1 + 0.549} = 0.646$.
- 2 $\hat{y}_2 = \sigma(-0.02) = \frac{1}{1 + e^{0.02}} = \frac{1}{1 + 1.020} = 0.495$.

z is almost zero, so the output is almost 0.5: the network is unsure

So $a^{[2]} = \hat{y} = (0.646, 0.495)^\top$. Forward propagation complete. Nova says "my name" with probability 0.646 (target 1) and "a question" with probability 0.495 (target 0). Both are on the right side of 0.5, but only just.



Step 4: sigmoid squashes to (0,1) → $\hat{y} = (0.646, 0.495)$. Compare with $y = (1, 0)$: loss 0.560.

Figure 5.5 Step 4 and the loss: $\hat{y} = (0.646, 0.495)$ is compared with $y = (1, 0)$ in the loss node L .

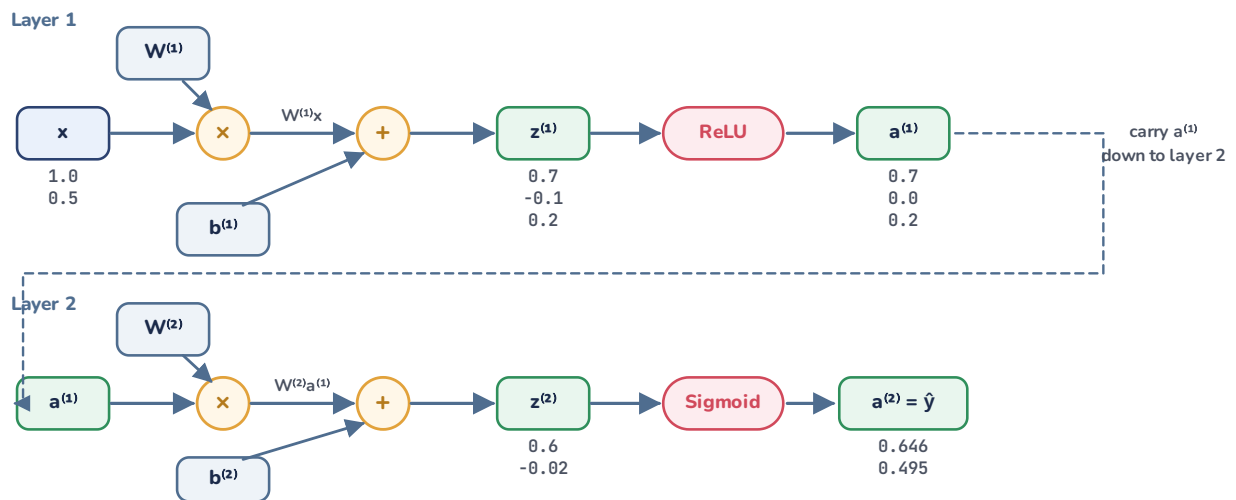


Figure 5.6 The forward pass as a computational graph, as on your slide: multiply, add, bend, twice. Every box's value is kept for the backward pass.

The loss

Each output is a yes/no answer with its own binary cross-entropy. The deck averages the two (so $n = 2$):

$$L = -\frac{1}{2} \sum_{i=1}^2 [y_i \log(a_i^{[2]}) + (1 - y_i) \log(1 - a_i^{[2]})]$$

COMPUTING L

- 1 Output 1 has $y_1 = 1$, so only the first term survives: $1 \cdot \log(0.646) + 0 \cdot \log(0.354) = \log(0.646) = -0.437$.
- 2 Output 2 has $y_2 = 0$, so only the second term survives: $0 \cdot \log(0.495) + 1 \cdot \log(1 - 0.495) = \log(0.505) = -0.683$.
- 3 Add and average with the minus sign: $L = -\frac{1}{2}[-0.437 + (-0.683)] = -\frac{1}{2}(-1.120) = 0.560$.
a loss of 0 would mean both outputs were exactly right

Backward pass

BACKWARD STEP 1: OUTPUT ERROR $\Delta^{[2]} = \partial L / \partial Z^{[2]} = A^{[2]} - Y$

Chapter 4 showed that for sigmoid + BCE the error signal at the output is prediction minus target.

- 1 $\delta_1^{[2]} = 0.646 - 1 = -0.354$.
negative: $\hat{y}_1$ is too low, $z_1^{[2]}$ must go up
- 2 $\delta_2^{[2]} = 0.495 - 0 = 0.495$.
positive: $\hat{y}_2$ is too high, $z_2^{[2]}$ must go down

So $\delta^{[2]} = (-0.354, 0.495)^\top$.

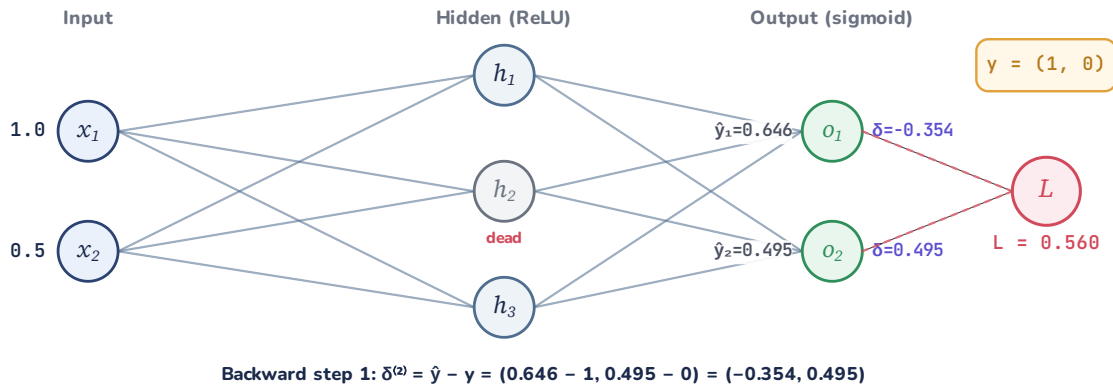


Figure 5.7 Backward step 1: the error signal beside each output, prediction minus target. The signs say which way each unit must move.

! CAREFUL: WHERE DID THE $\frac{1}{2}$ GO?

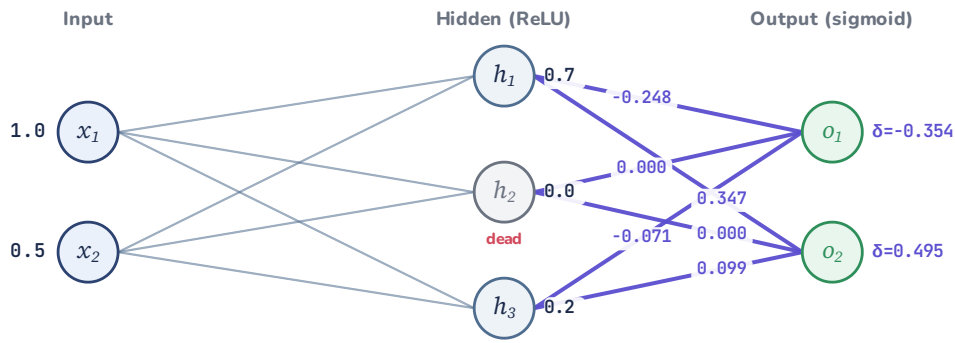
The loss averages the two outputs, $L = -\frac{1}{2}[\dots]$, so the exact gradient of L is $\frac{1}{2}(a^{[2]} - y) = (-0.177, 0.248)$. The deck uses $\delta^{[2]} = a^{[2]} - y$, which is the gradient of the *sum* of the two losses. Both are fine: they point in exactly the same direction, and the factor $\frac{1}{2}$ only scales every gradient, which the learning rate absorbs ($\alpha = 0.1$ on the sum is the same as $\alpha = 0.2$ on the average). We follow the deck's numbers from here on. In an exam, say which convention you use and stick to it.

BACKWARD STEP 2: GRADIENTS OF THE OUTPUT LAYER

Each weight's gradient is "error at the unit it feeds" times "the activation it multiplied". In column form that is the outer product $\delta^{[2]}(a^{[1]})^\top$, a 2×3 matrix, the same shape as $W^{[2]}$. Recall $a^{[1]} = (0.7, 0.0, 0.2)^\top$.

- 1 Row 1 (output 1): $-0.354 \times (0.7, 0.0, 0.2) = (-0.248, 0.000, -0.071)$.
- 2 Row 2 (output 2): $0.495 \times (0.7, 0.0, 0.2) = (0.347, 0.000, 0.099)$.
the middle column is 0 because the dead unit's activation is 0: a weight on a zero input cannot be blamed
- 3 Bias gradient: the bias multiplies a constant 1, so $\partial L / \partial b^{[2]} = \delta^{[2]} = (-0.354, 0.495)^\top$.

$$\frac{\partial L}{\partial W^{[2]}} = \begin{pmatrix} -0.354 \\ 0.495 \end{pmatrix} \begin{pmatrix} 0.7 & 0.0 & 0.2 \end{pmatrix} = \begin{pmatrix} -0.248 & 0.000 & -0.071 \\ 0.347 & 0.000 & 0.099 \end{pmatrix}, \quad \frac{\partial L}{\partial b^{[2]}} = \begin{pmatrix} -0.354 \\ 0.495 \end{pmatrix}$$



Backward step 2: gradient on each output arrow = δ (at its end) $\times a$ (at its start). $\partial L / \partial b^{(2)} = \delta^{(2)}$

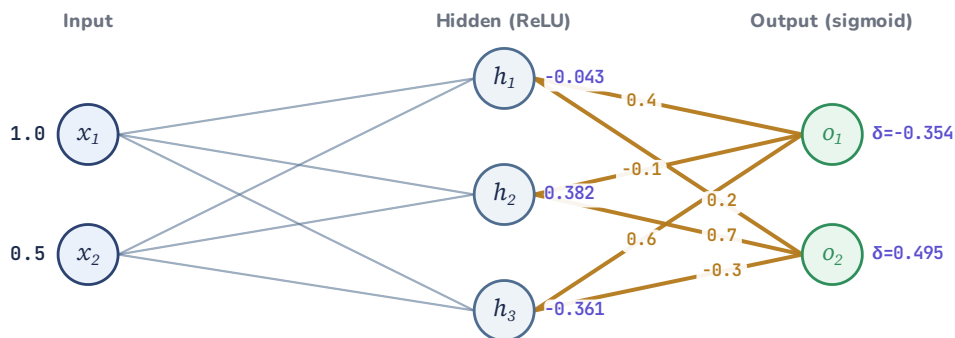
Figure 5.8 Backward step 2: each output arrow carries its gradient, the δ at its head times the a at its tail. The dead unit's arrows carry 0.000.

BACKWARD STEP 3: PASS THE ERROR BACK, $(W^{[2]})^\top \Delta^{[2]}$

Each hidden unit collects blame from both outputs, weighted by the arrow it sent to each. Transposing $W^{[2]}$ turns "row per output" into "row per hidden unit".

- 1 Unit 1: $0.4 \times (-0.354) + 0.2 \times 0.495 = -0.142 + 0.099 = -0.043$.
column 1 of $W^{[2]}$ is $(0.4, 0.2)$: unit 1's two outgoing weights
- 2 Unit 2: $-0.1 \times (-0.354) + 0.7 \times 0.495 = 0.035 + 0.347 = 0.382$.
- 3 Unit 3: $0.6 \times (-0.354) + (-0.3) \times 0.495 = -0.212 - 0.149 = -0.361$.

So $(W^{[2]})^\top \delta^{[2]} = (-0.043, 0.382, -0.361)^\top$.



Backward step 3: send $\delta^{(2)}$ back along the arrows $\rightarrow (W^{(2)})^\top \delta^{(2)} = (-0.043, 0.382, -0.361)$

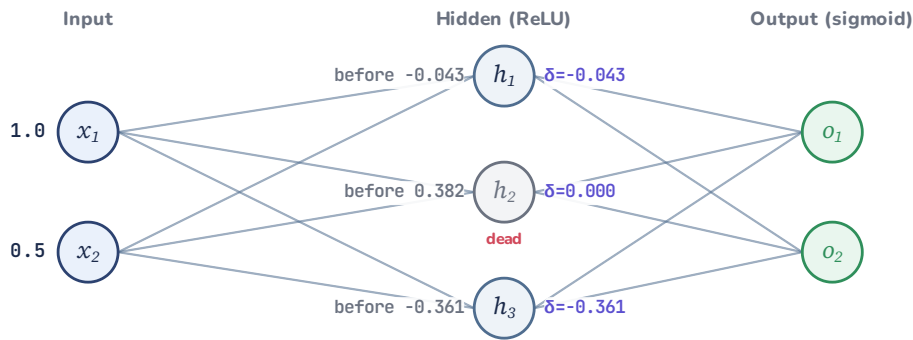
Figure 5.9 Backward step 3: the output errors travel back along the same arrows. Each hidden unit shows the blame it collected, before the ReLU slope.

BACKWARD STEP 4: MULTIPLY BY THE RELU SLOPES, $\Delta^{[1]} = (W^{[2]})^\top \Delta^{[2]} \odot G'(Z^{[1]})$

The slope of ReLU is 1 where $z > 0$ and 0 where $z \leq 0$. From step 1, $z^{[1]} = (0.7, -0.1, 0.2)$.

- 1 Slopes: $g'(z^{[1]}) = (1, 0, 1)^\top$, because $0.7 > 0, -0.1 \leq 0, 0.2 > 0$.
- 2 Multiply entry by entry: $\delta^{[1]} = (-0.043 \times 1, 0.382 \times 0, -0.361 \times 1) = (-0.043, 0.000, -0.361)$.

the dead unit's blame is thrown away: it had no effect on the output, so it cannot be to blame



Backward step 4: multiply by $\text{ReLU}'(z^{(2)}) = (1, 0, 1)$. The dead unit gets $\delta = 0$: no update.

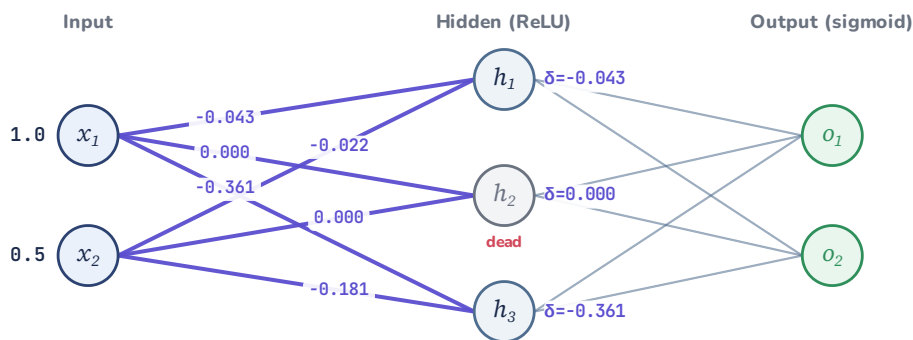
Figure 5.10 Backward step 4: left, the blame that arrived; right, the error signal after the ReLU slope. Unit 2 received 0.382 and keeps none: "Dead neuron gradient = 0 (no update!)".

BACKWARD STEP 5: GRADIENTS OF THE HIDDEN LAYER

Same rule as step 2, one layer earlier: $\partial L / \partial W^{[1]} = \delta^{[1]} x^\top$, a 3×2 matrix, with $x = (1.0, 0.5)^\top$.

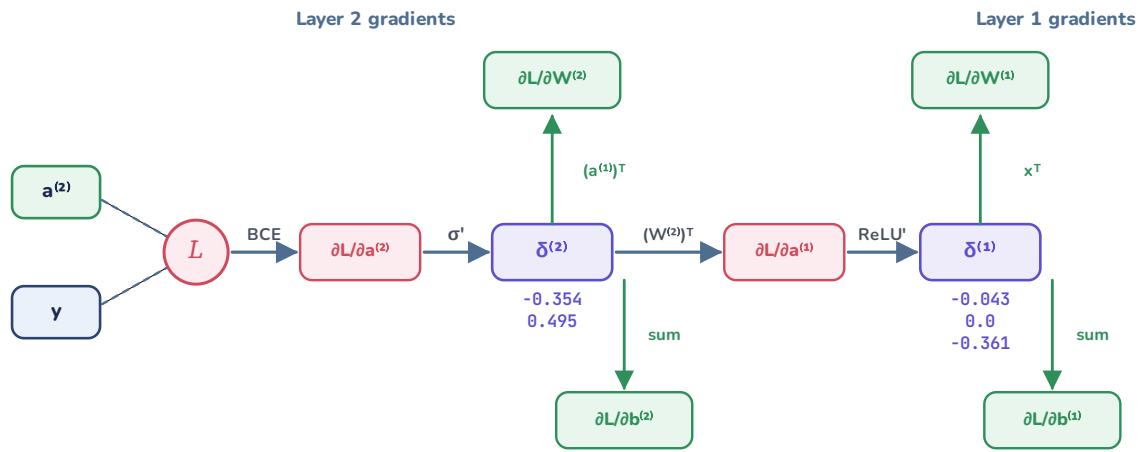
- 1 Row 1: $-0.043 \times (1.0, 0.5) = (-0.043, -0.022)$.
- 2 Row 2: $0.000 \times (1.0, 0.5) = (0.000, 0.000)$.
the dead unit's weights get zero gradient
- 3 Row 3: $-0.361 \times (1.0, 0.5) = (-0.361, -0.181)$.
- 4 Bias gradient: $\partial L / \partial b^{[1]} = \delta^{[1]} = (-0.043, 0.000, -0.361)^\top$.

$$\frac{\partial L}{\partial W^{[1]}} = \begin{pmatrix} -0.043 \\ 0.000 \\ -0.361 \end{pmatrix} \begin{pmatrix} 1.0 & 0.5 \end{pmatrix} = \begin{pmatrix} -0.043 & -0.022 \\ 0.000 & 0.000 \\ -0.361 & -0.181 \end{pmatrix}, \quad \frac{\partial L}{\partial b^{[1]}} = \begin{pmatrix} -0.043 \\ 0.000 \\ -0.361 \end{pmatrix}$$



Backward step 5: gradient on each input arrow = δ (at its end) $\times x$ (at its start). $\partial L / \partial b^{(1)} = \delta^{(1)}$

Figure 5.11 Backward step 5: each input arrow carries its gradient, the δ at its head times the x at its tail. The dead unit's arrows carry 0.000.



Read right-to-left along the middle row: the loss, then $\delta^{(2)}$, then $\delta^{(1)}$. Each δ feeds one weight gradient and one bias gradient.

Figure 5.12 The backward pass as a computational graph, as on your slide: loss $\rightarrow \partial L / \partial a^{[2]} \rightarrow$ (sigmoid slope) $\delta^{[2]} \rightarrow (W^{[2]})^T$, ReLU slope) $\delta^{[1]}$. Each δ branches up to a weight gradient and down to a bias gradient.

Update

The deck's rule, with learning rate $\alpha = 0.1$: $\theta_{\text{new}} = \theta_{\text{old}} - \alpha \frac{\partial L}{\partial \theta}$ for every parameter θ .

LAYER 2: $W_{\text{NEW}}^{[2]}$ AND $B_{\text{NEW}}^{[2]}$

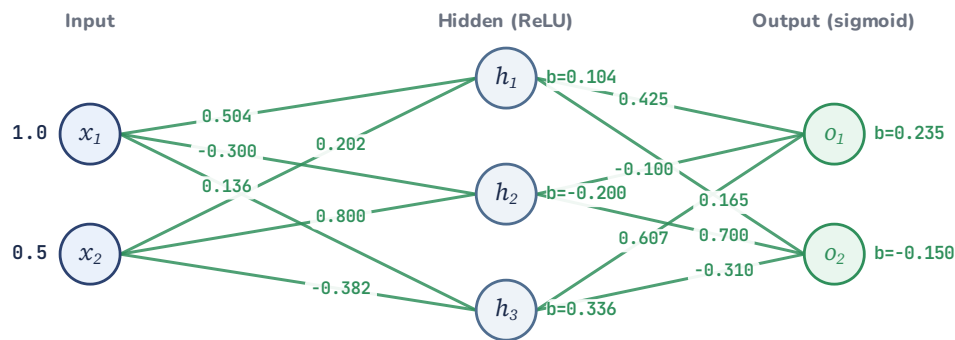
- 1 Scale the gradient: $0.1 \times \frac{\partial L}{\partial W^{[2]}} = \begin{pmatrix} -0.025 & 0.000 & -0.007 \\ 0.035 & 0.000 & 0.010 \end{pmatrix}$.
- 2 Subtract, entry by entry: row 1: $(0.4 + 0.025, -0.1 + 0.000, 0.6 + 0.007) = (0.425, -0.100, 0.607)$.
minus a negative is plus: weights with negative gradient grow
- 3 Row 2: $(0.2 - 0.035, 0.7 + 0.000, -0.3 - 0.010) = (0.165, 0.700, -0.310)$.
- 4 Biases: $b_{\text{new}}^{[2]} = (0.2 - 0.1 \times (-0.354), -0.1 - 0.1 \times 0.495) = (0.2 + 0.035, -0.1 - 0.050) = (0.235, -0.150)$.

$$W_{\text{new}}^{[2]} = \begin{pmatrix} 0.425 & -0.100 & 0.607 \\ 0.165 & 0.700 & -0.310 \end{pmatrix}, \quad b_{\text{new}}^{[2]} = \begin{pmatrix} 0.235 \\ -0.150 \end{pmatrix}$$

LAYER 1: $W_{\text{NEW}}^{[1]}$ AND $B_{\text{NEW}}^{[1]}$

- 1 Scale the gradient: $0.1 \times \frac{\partial L}{\partial W^{[1]}} = \begin{pmatrix} -0.004 & -0.002 \\ 0.000 & 0.000 \\ -0.036 & -0.018 \end{pmatrix}$.
- 2 Row 1: $(0.5 + 0.004, 0.2 + 0.002) = (0.504, 0.202)$.
- 3 Row 2: $(-0.3 + 0.000, 0.8 + 0.000) = (-0.300, 0.800)$.
the dead unit does not move at all
- 4 Row 3: $(0.1 + 0.036, -0.4 + 0.018) = (0.136, -0.382)$.
- 5 Biases: $b_{\text{new}}^{[1]} = (0.1 + 0.004, -0.2 + 0.000, 0.3 + 0.036) = (0.104, -0.200, 0.336)$.

$$W_{\text{new}}^{[1]} = \begin{pmatrix} 0.504 & 0.202 \\ -0.300 & 0.800 \\ 0.136 & -0.382 \end{pmatrix}, \quad b_{\text{new}}^{[1]} = \begin{pmatrix} 0.104 \\ -0.200 \\ 0.336 \end{pmatrix}$$



After the update ($\alpha = 0.1$): every number moved a little against its gradient. Row 2 of $W^{(1)}$ did not move.

Figure 5.13 After the update. Compare with Figure 5.1: everything moved a little, except the dead unit's two weights and bias. Unit 3's arrows moved most: they carried the biggest error signal.



new = old - $\eta \times$ gradient, for every weight matrix and every bias vector

Figure 5.14 The update as a computational graph: for each parameter block, scale the gradient by η (the deck's α) and subtract.

Did it help?

Run the forward pass once more with the new parameters (same input, same target). One line per number, as before:

- 1 $z^{[1]} = W_{\text{new}}^{[1]}x + b_{\text{new}}^{[1]} = (0.709, -0.100, 0.281)$; $a^{[1]} = (0.709, 0.000, 0.281)$.
unit 2 is still dead: it did not move, so its sum is still -0.1
- 2 $z^{[2]} = W_{\text{new}}^{[2]}a^{[1]} + b_{\text{new}}^{[2]} = (0.707, -0.120)$; $\hat{y} = (\sigma(0.707), \sigma(-0.120)) = (0.670, 0.470)$.
- 3 $L = -\frac{1}{2}[\log 0.670 + \log(1 - 0.470)] = -\frac{1}{2}[-0.401 - 0.635] = 0.518$.
down from 0.560

Output 1 rose from 0.646 towards its target 1; output 2 fell from 0.495 towards its target 0. The loss dropped from 0.560 to 0.518. One step, one small improvement. Training is this, repeated.

Σ MATHS, STEP BY STEP: THE SAME STEP IN MODULE 5'S ROW NOTATION

COLUMN FORM (THIS CHAPTER)

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]} \text{ (} W^{[2]} \text{ is } 2 \times 3\text{); } \delta^{[2]} = a^{[2]} - y \text{ (} 2 \times 1\text{); } \partial L / \partial W^{[2]} = \delta^{[2]}(a^{[1]})^\top \text{ (} 2 \times 3\text{); } \delta^{[1]} = ((W^{[2]})^\top \delta^{[2]}) \odot g'(z^{[1]}) \text{ (} 3 \times 1\text{); } \partial L / \partial W^{[1]} = \delta^{[1]}x^\top \text{ (} 3 \times 2\text{).}$$

→

ROW FORM (MODULE 5, B = 1)

$$z^{(2)} = \mathbf{h}^{(1)}\mathbf{W}^{(2)} + \mathbf{b}^{(2)} \text{ with } \mathbf{W}^{(2)} = (W^{[2]})^\top \text{ (} 3 \times 2\text{); } \delta^{(2)} = \hat{\mathbf{y}} - \mathbf{y} = (-0.354, 0.495) \text{ (} 1 \times 2\text{); } \partial J / \partial \mathbf{W}^{(2)} = \mathbf{h}^{(1)\top} \delta^{(2)} \text{ (} 3 \times 2\text{, the transpose of the column form, same six numbers); } \delta^{(1)} = (\delta^{(2)}\mathbf{W}^{(2)\top}) \odot \sigma'(\mathbf{z}^{(1)}) = (-0.043, 0, -0.361); \partial J / \partial \mathbf{W}^{(1)} = \mathbf{x}^\top \delta^{(1)} \text{ (} 2 \times 3\text{). Same numbers, lying down instead of standing up.}$$

📄 ON YOUR SLIDES

Slides 3–4: the problem and the initial network. Slides 5–12: each forward step as a "Solution" slide plus a picture ($z^{[1]}$, ReLU with "Neuron 2 is dead", $z^{[2]}$, sigmoid). Slide 13: forward graph. Slides 15–16: BCE 0.560; 17–18: $\delta^{[2]} = a^{[2]} - y$; 19–20: output gradients; 21–22: $(W^{[2]})^\top \delta^{[2]}$; 23–24: ReLU derivative, "Dead neuron gradient = 0 (no update!)"; 25–26: hidden gradients; 27: backward graph. Slides 28–30: the update with $\alpha = 0.1$; 31: update graph. "Did it help?" is not on the slides.

🔄 TRY IT YOURSELF

Redo the update with $\alpha = 0.5$ for $b^{[2]}$ only. What is $b_{\text{new}}^{[2]}$? $((0.2 + 0.177, -0.1 - 0.248) = (0.377, -0.348)$: five times the move. A bigger learning rate takes bigger steps; too big, and it overshoots.)

✅ CHECK YOURSELF

1. Why is $a_2^{[1]} = 0$? **Its sum $z_2^{[1]} = -0.1$ is negative and ReLU turns negatives into 0.**
2. What is $\delta^{[2]}$ and where does the formula come from? $a^{[2]} - y = (-0.354, 0.495)$; **sigmoid + BCE gives "prediction minus target".**
3. Why is the middle column of $\partial L / \partial W^{[2]}$ zero? **Those weights multiply $a_2^{[1]} = 0$; a weight on a zero input has no effect, so no gradient.**
4. Why is row 2 of $\partial L / \partial W^{[1]}$ zero? **$\delta_2^{[1]} = 0$ because the ReLU slope at $z = -0.1$ is 0. The dead unit gets no update.**
5. What happened to the loss after the update? **It fell from 0.560 to 0.518.**

👉 IN ONE BREATH

- Forward: $z^{[1]} = (0.7, -0.1, 0.2)$, $a^{[1]} = (0.7, 0, 0.2)$ (unit 2 dead), $z^{[2]} = (0.6, -0.02)$, $\hat{\mathbf{y}} = (0.646, 0.495)$. Loss 0.560.
- Backward: $\delta^{[2]} = \hat{\mathbf{y}} - \mathbf{y} = (-0.354, 0.495)$; $\partial L / \partial W^{[2]} = \delta^{[2]}(a^{[1]})^\top$; back through $(W^{[2]})^\top$ then $\odot$ ReLU' gives $\delta^{[1]} = (-0.043, 0, -0.361)$; $\partial L / \partial W^{[1]} = \delta^{[1]}x^\top$.
- Update: new = old $- 0.1 \times$ gradient, for all 17 parameters. The dead unit does not move.
- Loss after the step: $0.518 < 0.560$. Repeat.

Gradient flow, depth, and designing a network

Why deep sigmoid networks stop learning, why depth is still worth it, and how to choose the number of layers and units for a dataset.

On your slides: **Numerical deck, slides 33–41** · Gradient flow analysis · Design a neural network (three examples) · **Module 5, slides 44–53** · Depth vs width · Why depth matters · Universal approximation · Challenges of very deep networks · Design considerations · Output layer design · Architecture guidelines · Hidden activations · Regularisation

STORY

Asha builds Nova a four-layer network with sigmoid everywhere, because sigmoid is what she learned first. It trains, slowly. Nova's screen shows the size of the error signal reaching each layer: 1.0, 0.2, 0.04, 0.008. Ravi: "So the layer that touches the sound is the one that never learns?" Asha does the sum, switches to ReLU, and asks the next question: how many layers and units should Nova have, for the clips she actually has?

BIG IDEA

Every backward step through a layer multiplies the error signal by the size of that layer's weights and by the slope of its activation. For sigmoid the slope is at most 0.25, so with $\|W\| = 0.8$ each step shrinks the signal to 0.2 of what it was: the **vanishing gradient**. Depth is still worth having, because deep networks build features in a hierarchy and can represent far more with the same number of parameters. Design means choosing L , the n_ℓ , the activations, the output layer and the regularisation. Two rough rules size a network to the data you have.

THINK OF IT LIKE...

A message passed back along a corridor of closed doors. Each door lets through only a fifth of the sound. After one door the message is at 20%, after two at 4%, after three at 0.8%: a whisper nobody can act on. The doors are the layers; how much each door lets through is $\|W\| \times \sigma'$. ReLU is a door held open: on the positive side its slope is exactly 1, so the message passes unchanged.

Gradient flow: how much signal reaches each layer?

The Numerical deck's problem: a 4-layer network with sigmoid activations. Every weight matrix has size $\|W^{[l]}\| = 0.8$ (its spectral norm, the most it can stretch a vector). Every sigmoid slope is at its maximum, $g'^{[l]} = 0.25$. The error signal at the output has size $|\delta^{[4]}| = 1.0$. How big is the signal at each earlier layer?

PLAIN MATHS

From Chapter 4, going back one layer multiplies by the weights and by the slope.

Taking sizes:

$$|\delta^{[l-1]}| = |\delta^{[l]}| \cdot \|W^{[l]}\| \cdot |g'^{[l-1]}|$$

Each step multiplies by $0.8 \times 0.25 = 0.2$.

Layer by layer:

$$|\delta^{[3]}| = 1.0 \times 0.8 \times 0.25 = 0.2$$

$$|\delta^{[2]}| = 0.2 \times 0.8 \times 0.25 = 0.04$$

$$|\delta^{[1]}| = 0.04 \times 0.8 \times 0.25 = 0.008$$

After three backward steps the signal is 0.8% of what it was.

MATRIX FORM

The backprop equation $\delta^{(\ell)} = (\delta^{(\ell+1)} W^{(\ell+1)\top}) \odot \sigma'(\mathbf{Z}^{(\ell)})$ gives, for the sizes of the vectors,

$$\|\delta^{(\ell)}\| \leq \|\delta^{(\ell+1)}\| \cdot \|W^{(\ell+1)}\| \cdot \max \sigma'$$

Apply it $L - 1$ times:

$$\|\delta^{(1)}\| \leq \|\delta^{(L)}\| \cdot (\|W\| \max \sigma')^{L-1}$$

If the factor in brackets is below 1, the signal shrinks *exponentially* with depth. With sigmoid, $\max \sigma' = 0.25$, so the factor is below 1 unless the weights are bigger than 4. With ReLU, $\max \sigma' = 1$, and the factor is just $\|W\|$: with 0.8 the same chain gives $0.8^3 = 0.512$, not 0.008.

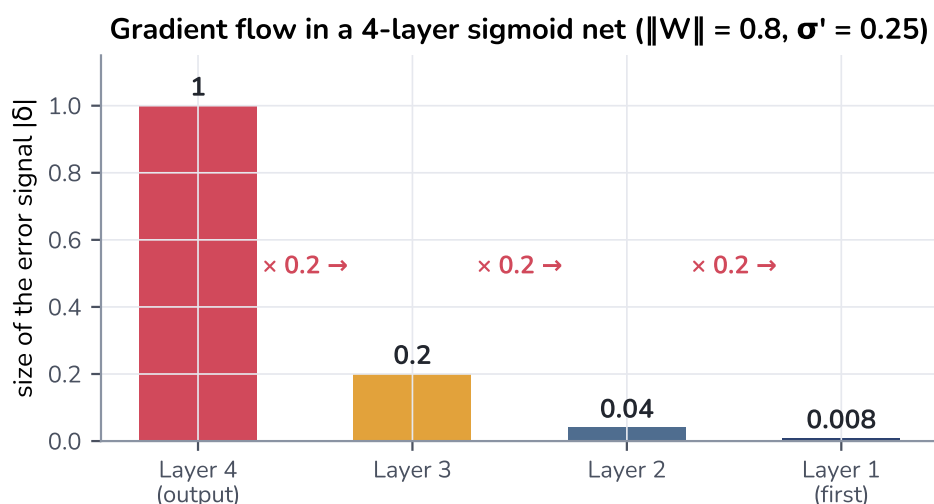


Figure 6.1 The error signal at each layer of the deck's 4-layer sigmoid network. Each bar is one fifth of the one before. Notice that layer 1, the one that touches the raw input, gets 0.008: almost no learning signal at all. That is the vanishing gradient problem.

The deck's conclusions: a factor of 0.2 per layer; 0.8% after three steps; the first layer receives an extremely weak learning signal: the vanishing gradient problem in deep sigmoid networks. The cure: ReLU in the hidden layers, plus careful initialisation and normalisation (Chapter 7).

Depth versus width

There are two ways to give a network more capacity: more layers (**depth**) or more units per layer (**width**).

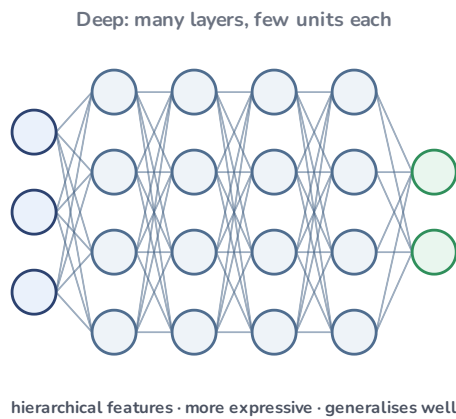


Figure 6.2a Deep: hierarchical features, more expressive, better generalisation.

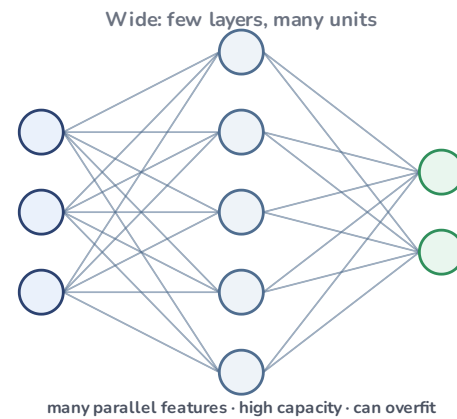


Figure 6.2b Wide: many parallel features, higher capacity, but a bigger risk of overfitting.

WHY DEPTH MATTERS

Deep networks learn a **hierarchy**: simple things in the first layer, bigger things built from them in the next. Your slide's example is image classification: layer 1 detects edges and colours, layer 2 combines them into textures and simple shapes, layer 3 recognises object parts (eyes, wheels), layer 4 and up identify whole objects (faces, cars). And a deep network can represent *exponentially more functions* than a shallow one with the same number of parameters, because each layer reuses everything the layer below it built.

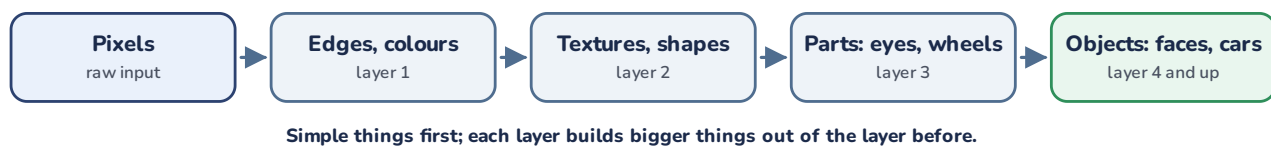


Figure 6.3 The hierarchy of features on your slide. Each layer only has to learn one small step up. Notice that "eyes" are built from "curves", not from pixels; the earlier layers did that work already.

UNIVERSAL APPROXIMATION THEOREM (CYBENKO 1989, HORNIK ET AL. 1989), IN PLAIN WORDS

A feed-forward network with *one* hidden layer, enough units and a suitable activation can approximate any continuous function as closely as you like on any bounded region of inputs. So in theory one hidden layer is enough. **But** "enough units" may be exponentially many: a wide layer builds every feature from scratch, while a deep network builds edges once and reuses them for every eye, wheel and letter. In practice deep networks learn better representations, need fewer parameters, and generalise better. Depth is the parameter-efficient way to get the same power.

Challenges of very deep networks

Problem	What happens	Solutions on your slide
Vanishing gradients	gradients become exponentially small in early layers; the network fails to learn useful features	ReLU activations, careful initialisation, normalisation
Exploding gradients	gradients become exponentially large; numbers blow up to NaN or Inf	gradient clipping, proper initialisation

Problem	What happens	Solutions on your slide
Degradation	very deep networks are harder to optimise; training error can <i>rise</i> with depth	residual connections (ResNets), skip connections — Book 6

Designing a network

Your slide lists five decisions: (1) the number of layers L , how deep; (2) the units per layer $n_1, \dots, n_L$, how wide; (3) the activation $\sigma^{(\ell)}$ of each layer; (4) the output layer, set by the task (table below); (5) regularisation, to stop overfitting. There is no universal answer: it depends on the problem, the data and the computer you have.

Task	Output units K	Output activation	Loss
Regression (continuous values)	the dimension of the output	identity (linear)	MSE
Binary classification	1	sigmoid $\sigma(z) = 1/(1 + e^{-z})$	BCE
Multi-class, C classes	C	softmax $e^{z_k} / \sum_j e^{z_j}$	CCE

GENERAL GUIDELINES

- **Start simple.** Begin with 2–3 hidden layers of 10–100 units each, depending on the problem's scale. Add complexity only if needed.
- **Common shapes.** Decreasing width $n_1 > n_2 > \dots$: a *funnel* that compresses. Constant width $n_1 = n_2 = \dots = n_{L-1}$: a uniform representation. *Hourglass*, wide $\rightarrow$ narrow $\rightarrow$ wide: learns a compressed representation in the middle.
- **Scale with data.** More data can support a larger network; less data means keep the network small to avoid overfitting.

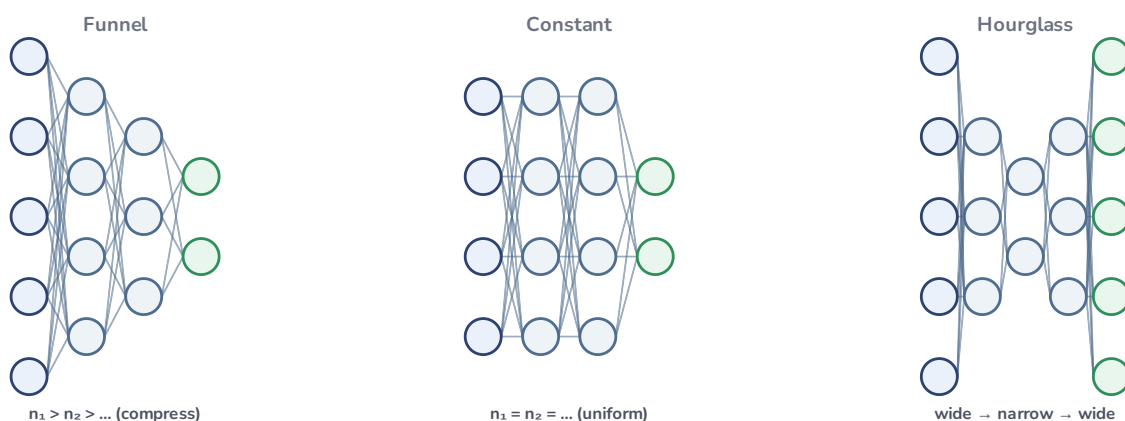


Figure 6.4 The three common shapes on your slide: funnel, constant, hourglass.

Hidden activation	When to use	Avoid when
ReLU	default choice; fast, no vanishing gradient	very deep networks with many dying units
Leaky ReLU	deep networks; fixes dying ReLU	never — a safe choice
Tanh	small networks; zero-centred	very deep networks (vanishing gradient)
Sigmoid	only for a binary output	hidden layers (strong saturation)

Modern recommendation: ReLU for hidden layers by default; Leaky ReLU or ELU for very deep networks; avoid sigmoid and tanh in hidden layers unless you have a specific reason. To prevent overfitting, four techniques are named on your slide, and taught in Modules 10 and 11: **L2**

regularisation (weight decay), dropout, early stopping, and batch normalisation.

How big? Sizing a network to the data

The Numerical deck gives two rules of thumb. With M training samples, d input features and c classes:

$$L \leq \log_2\left(\frac{M}{d \times c}\right) + 1, \quad \text{Parameters} \leq \frac{M}{5}$$

The first limits depth by how much data you have per input-and-class. The second says: at least five training samples for every parameter you want to fit. Here are the deck's three examples, every count recomputed. (Remember $n_\ell(n_{\ell-1} + 1)$ per layer.)

EXAMPLE 1: SMALL TABULAR DATASET — $D = 20$, $M = 800$, $C = 3$

- 1 $L \leq \log_2(800/(20 \times 3)) + 1 = \log_2(13.33) + 1 \approx 3.7 + 1 = 4.7$, so $L \leq 4$. Budget: $800/5 = 160$ parameters.
- 2 $M < 1000$, so start with $L = 2$: $[20, 40, 3]$. Parameters = $20 \times 40 + 40 + 40 \times 3 + 3 = 800 + 40 + 120 + 3 = 963$. $963 > 160$: too many.
- 3 Try $[20, 20, 3]$: $20 \times 20 + 20 + 20 \times 3 + 3 = 483$. Still too many.
- 4 Try $[20, 10, 3]$: $20 \times 10 + 10 + 10 \times 3 + 3 = 243$. Still too many.
- 5 Try $[20, 6, 3]$: $20 \times 6 + 6 + 6 \times 3 + 3 = 120 + 6 + 18 + 3 = 147 \leq 160$. ✓
final recommendation: $[20, 6, 3]$, with regularisation

EXAMPLE 2: MEDIUM IMAGE DATASET — $D = 784$, $M = 5000$, $C = 10$

- 1 The depth rule gives $\log_2(5000/7840) + 1 \approx 0.35$: less than one layer. The deck takes a relaxed approach and starts with $L = 3$: $[784, 2352, 784, 10]$. Budget: $5000/5 = 1000$.
- 2 Parameters = $784 \times 2352 + 2352 + 2352 \times 784 + 784 + 784 \times 10 + 10 \approx 3.7$ million. Way too many.
- 3 Try $[784, 128, 64, 10]$: $784 \times 128 + 128 + 128 \times 64 + 64 + 64 \times 10 + 10 = 100,352 + 128 + 8,192 + 64 + 640 + 10 = 109,386$ (the slide rounds to 109,000). Still far over 1000.
- 4 Try $[784, 64, 32, 10]$: 52,650. $[784, 32, 10]$: 25,450. $[784, 16, 10]$: 12,730. All far over budget.
- 5 Reality check: with 784 inputs even the smallest useful network breaks the $M/5$ rule. The deck's answer: use a larger architecture *with regularisation* (dropout, L2) rather than shrink it into uselessness.

EXAMPLE 3: LARGE TEXT DATASET — $D = 1000$, $M = 50,000$, $C = 5$

- 1 $L \leq \log_2(50000/(1000 \times 5)) + 1 = \log_2(10) + 1 \approx 3.3 + 1 = 4.3$, so $L \leq 4$. Budget: $50000/5 = 10,000$.
- 2 $M \geq 10,000$, so $L = 4$ is allowed: $[1000, 5000, 2000, 1000, 5]$. Parameters: $1000 \times 5000 + 5000 \approx 5\text{M}$, $5000 \times 2000 + 2000 \approx 10\text{M}$, $2000 \times 1000 + 1000 \approx 2\text{M}$, $1000 \times 5 + 5 \approx 5\text{K}$. Total $\approx 17\text{M}$. Way too many.

- 3 Try [1000, 512, 256, 128, 5]: $512,512 + 131,328 + 32,896 + 645 = 677,381$, about 675K on the slide (it rounds each part down). ✓
- 4 That is $677,381/50,000 \approx 13.5$ parameters per sample, which the deck calls reasonable. Final recommendation: [1000, 512, 256, 128, 5] with batch normalisation and dropout. a funnel: 1000 → 512 → 256 → 128 → 5

! CAREFUL

These rules are rough guides, not laws: Example 2 breaks the depth rule outright, and all three answers lean on regularisation. Use them for a first size, then let the validation loss (Chapter 7) decide. Two slide arithmetic slips: [784, 64, 32, 10] is 52,650 (slide: 52,458) and [784, 16, 10] is 12,730 (slide: 12,714); the conclusions do not change.

📄 ON YOUR SLIDES

Numerical deck: slides 33–34 the gradient-flow problem, its formula and the values 1.0, 0.2, 0.04, 0.008; slides 36–41 the three design examples with the two rules. Module 5: slide 44 depth vs width; 45 why depth matters; 46 the Universal Approximation Theorem; 47 the three challenges; 49 design considerations; 50 output layer design; 51 general guidelines; 52 hidden activations; 53 regularisation, "Module 10 and 11".

✓ CHECK YOURSELF

1. By what factor does the error signal shrink per layer in the deck's sigmoid example, and why? $0.8 \times 0.25 = 0.2$: **weight size times the maximum sigmoid slope.**
2. What fraction of the signal reaches layer 1 after three steps? **0.008, i.e. 0.8%.**
3. What does the Universal Approximation Theorem promise, and what does it not promise? **One hidden layer can approximate any continuous function on a bounded region; it does not promise a small number of units.**
4. Name the three challenges of very deep networks and one fix each. **Vanishing (ReLU/init/normalisation), exploding (clipping/init), degradation (residual connections).**
5. Size a network for $d = 50$, $M = 2000$, $c = 2$: does [50, 6, 2] fit the budget? **Budget 400; $300 + 6 + 12 + 2 = 320$, yes. [50, 8, 2] would be 426, no.**

👉 IN ONE BREATH

- Backward through a sigmoid layer: $\text{signal} \times \|W\| \times \sigma' \leq 0.8 \times 0.25 = 0.2$. Four layers: $1.0 \rightarrow 0.2 \rightarrow 0.04 \rightarrow 0.008$. Vanishing gradient; use ReLU.
- Depth builds a hierarchy of features and is parameter-efficient; one wide layer can do anything in theory but may need exponentially many units.
- Very deep: vanishing, exploding, degradation → ReLU, good init, clipping, residual connections.
- Design: choose L , n_ℓ , activations (ReLU default), output layer by task, regularisation. Start with 2–3 layers of 10–100 units.
- Sizing rules: $L \leq \log_2(M/(dc)) + 1$, parameters $\leq M/5$; tabular [20,6,3]; images need regularisation; text [1000,512,256,128,5].

The training toolbox

How to start the weights, which knobs to set, how to prepare the data, what to watch while training, what to do when it goes wrong — and how a trained network answers a new question.

On your slides: **Module 5, slides 55–61** · Parameter initialisation · Training hyperparameters · Feature preprocessing · Monitoring training progress · Diagnosing training issues · Debugging checklist · Implementation best practices · **slides 41–42** · Prediction and batch prediction

STORY

Asha moves from the whiteboard to her laptop and trains Nova on all 500 clips. First run: the loss shows "NaN" after ten steps. Second run: the loss sits at 0.69 and never moves. Third run: the training loss falls beautifully and Nova still wakes up when a door slams. Ravi reads the three screens. "Three different diseases." Asha opens her slides. There is a table for exactly this.

BIG IDEA

Training is a recipe with settings. Start the weights **randomly** (never all zero). Set the **learning rate**, **batch size** and **number of epochs** sensibly and use early stopping. **Standardise** the inputs, with the same transformation on validation and test data. **Watch** the training and validation losses, and read the symptom table when something looks wrong. Once trained, using the network is just the forward pass.

THINK OF IT LIKE...

Cooking a dish for the first time. Prepare the ingredients so they are all cut to the same size (preprocessing). Start the flame at medium, not full (learning rate). Taste as you go, not only at the end (monitoring). Keep a card that says "too salty → add water; raw in the middle → lower the flame, cook longer" (the diagnosing table). And before the big dinner, cook one small portion to check the recipe works at all (overfit a small batch).

Where to start: initialisation

Before training, every weight needs a starting value. Your slide is blunt: **zero initialisation is bad; random initialisation is good**. If every weight in a layer starts at zero, every unit in that layer computes the same thing, gets the same gradient, and takes the same step. They stay identical forever. The layer behaves like a single unit.

Random starts break the tie. Two named recipes choose the *size* of the random numbers, so that signals neither vanish nor explode as they pass through the layers. **Xavier/Glorot initialisation** is for sigmoid and tanh; **He initialisation** is for ReLU. Biases start at zero: $\mathbf{b}^{(\ell)} = \mathbf{0}$. The details are in Modules 10 and 11.



Figure 7.1 Left: all-zero weights make every hidden unit a copy of the others, and they never become different. Right: small random weights let each unit start on its own path.

Three knobs to tune

1. **Learning rate η .** Start with one of $\{0.001, 0.01, 0.1\}$. Consider an adaptive method such as *Adam* with $\eta = 0.001$ (a later module).
2. **Batch size B .** Common: $\{32, 64, 128, 256\}$. Larger batches are faster per epoch but may generalise worse; smaller batches generalise better but are slower. GPU memory sets the upper limit.
3. **Number of epochs T .** Typically 50–200 for a moderate dataset. Use **early stopping**: stop when the validation loss stops improving. Watch for overfitting.

Prepare the inputs: preprocessing

Three things to do before training, from your slide.

- **Normalise or standardise.** Standardisation is recommended: subtract the mean, divide by the standard deviation, so each feature is centred at 0 with spread 1. Min-max scaling squeezes each feature into $[0, 1]$. Either way, **apply the same transformation to the validation and test sets**, using the mean and spread computed on the training set.
- **Handle missing values.** Fill them with the mean, median or mode of the feature, or add an indicator column that says "this value was missing".
- **Encode categories.** One-hot encoding for categories with no order (city names); ordinal encoding (0, 1, 2, ...) for categories with an order (small, medium, large).

WORKED EXAMPLE: STANDARDISING THREE AGES

Training ages: 20, 30, 40. A new test age: 50.

- 1 Mean = $(20 + 30 + 40)/3 = 30$. Standard deviation = $\sqrt{(100 + 0 + 100)/3} = \sqrt{66.7} = 8.165$.
- 2 Standardised: $(20 - 30)/8.165 = -1.225$, $(30 - 30)/8.165 = 0$, $(40 - 30)/8.165 = 1.225$.
centred at 0, spread 1
- 3 Min-max instead: $(20 - 20)/20 = 0$, $(30 - 20)/20 = 0.5$, $(40 - 20)/20 = 1$.
- 4 The test age uses the *training* mean and spread: $(50 - 30)/8.165 = 2.449$.
never recompute the mean on test data; the network learned with the training scale

Watch it train: monitoring

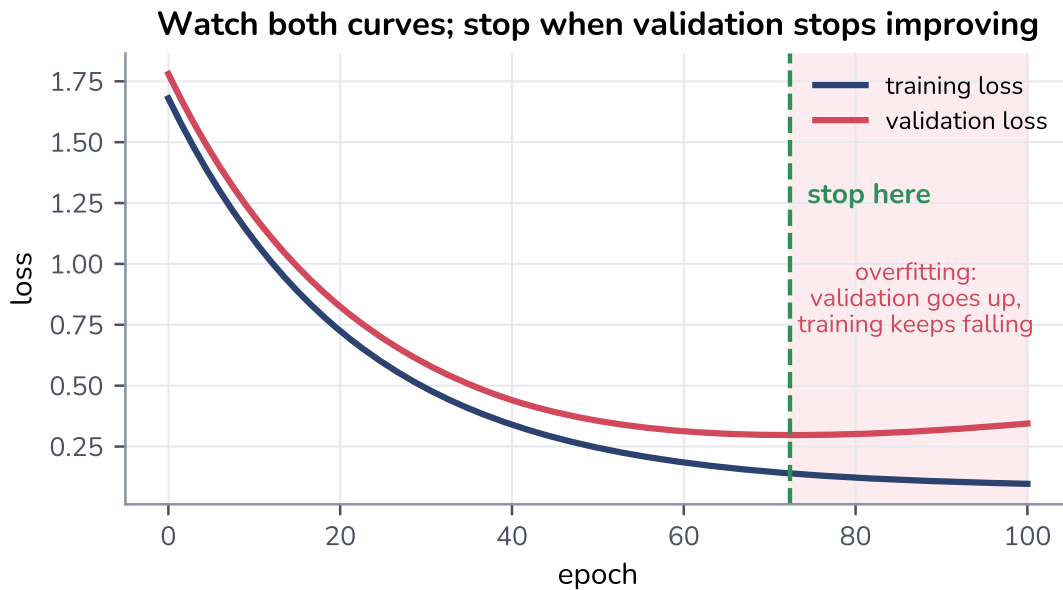


Figure 7.2 The picture on your slide. Training loss keeps falling. Validation loss falls, then flattens, then rises: the network has started memorising the training set. Stop at the dashed line, where validation loss is lowest. That is early stopping.

Track four things: the **training loss** (should decrease steadily); the **validation loss** (should decrease, then plateau or increase); training and validation accuracy for classification; and the **gradient norms**, to detect vanishing or exploding gradients early.

When it goes wrong: the diagnosing table

Symptom	Diagnosis	Solution
Loss = NaN or Inf	numerical instability	decrease η , check the gradients, normalise the inputs
Loss not decreasing	learning rate too small, or a bug	increase η , verify the gradient computation
Loss oscillating	learning rate too large	decrease η , use a learning-rate schedule
Training loss $\downarrow$, validation loss $\uparrow$	overfitting	add regularisation, dropout, more data
Both losses high	underfitting	increase capacity (wider or deeper), train longer
Slow convergence	poor initialisation or bad features	proper init (He / Xavier), normalise the features
Gradients $\rightarrow 0$	vanishing gradients	use ReLU, better init, batch normalisation
Gradients $\rightarrow \infty$	exploding gradients	gradient clipping, decrease η

Asha's three runs are rows 1, 2 and 4: a learning rate too large for unnormalised inputs, a learning rate too small (0.69 is log 2, the BCE loss of a network that always says 0.5), and overfitting.

The debugging checklist

Before training

features normalised · data split into train/val/test · output layer matches the task (sigmoid for binary, softmax for multi-class) · loss matches the output activation · weights initialised properly, not all zero

During training

training loss decreasing · validation loss tracked separately · gradients neither vanishing nor exploding · no NaN/Inf in loss or activations · training accuracy improving

Sanity checks

try to overfit a small batch: the loss should reach 0 · verify the shapes of all matrices · test the forward pass before adding backprop

BEST PRACTICES

- **Vectorise.** Always use matrix operations, never loops over examples; that is what lets the GPU help.
- **Modular code.** Separate functions for the forward pass, the loss and the backward pass; test each on its own.
- **Numerical stability.** Use stable formulas (log-sum-exp for softmax, Chapter 8); clip gradients when $\|\nabla\| > \text{a threshold}$; watch for NaN/Inf.
- **Reproducibility.** Set random seeds, save hyperparameters, track experiments systematically.

Using the trained network

After training, the weights are frozen. Prediction is nothing more than the forward pass of Chapter 4 on a new input.



Figure 7.3 Prediction. The new example goes through the same layers with the trained weights; the output is read according to the task.

Σ MATHS, STEP BY STEP

PLAIN MATHS (ONE NEW EXAMPLE)

Set $\mathbf{h}^{(0)} = \mathbf{x}_{\text{new}}$. For $\ell = 1, \dots, L$:

$$\mathbf{z}^{(\ell)} = \mathbf{h}^{(\ell-1)} \mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)}, \quad \mathbf{h}^{(\ell)} = \sigma^{(\ell)}(\mathbf{z}^{(\ell)})$$

Output $\hat{\mathbf{y}}_{\text{new}} = \mathbf{h}^{(L)}$. Then read it: *regression*: $\hat{\mathbf{y}}_{\text{new}}$ is the predicted value; *binary*: class 1 if $\hat{y}_{\text{new}} > 0.5$, else class 0; *multi-class*: the predicted class is $\arg \max_k \hat{y}_{k, \text{new}}$.

In Chapter 5, $\hat{\mathbf{y}} = (0.646, 0.495)$ reads as "my name: yes ($0.646 > 0.5$); a question: no ($0.495 < 0.5$)".

→

MATRIX FORM (A WHOLE TEST SET)

Stack all N_{test} new examples as rows of $\mathbf{X}_{\text{test}} \in \mathbb{R}^{N_{\text{test}} \times d}$:

$$\mathbf{H}^{(0)} = \mathbf{X}_{\text{test}}, \quad \mathbf{Z}^{(\ell)} = \mathbf{H}^{(\ell-1)} \mathbf{W}^{(\ell)} + \mathbf{1}_{N_{\text{test}}} \mathbf{b}^{(\ell)\top}, \quad \mathbf{H}^{(\ell)} = \sigma^{(\ell)}(\mathbf{Z}^{(\ell)})$$

$$\hat{\mathbf{Y}}_{\text{test}} = \mathbf{H}^{(L)} \in \mathbb{R}^{N_{\text{test}} \times K}$$

All predictions in one forward pass, one row each. Efficient matrix operations on a GPU, and essential when a model must answer millions of queries.

! CAREFUL

The validation set is for decisions (when to stop, which η); the test set is touched once, at the end, to report performance. And the preprocessing statistics (means, spreads, min and max) come from the training set only. Recomputing them on the test set leaks information and changes the scale the network was trained on.

📄 ON YOUR SLIDES

Slide 55: initialisation ("Bad: zero; Good: random", Xavier/Glorot, He, biases zero). Slide 56: learning rate, batch size, epochs. Slide 57: preprocessing. Slide 58: the loss-versus-epoch picture with "Overfitting" and "Stop here". Slide 59: the diagnosing table. Slide 60: the debugging checklist. Slide 61: best practices. Slides 41–42: prediction, equations (51)–(56).

✓ CHECK YOURSELF

1. Why is all-zero initialisation bad? **All units in a layer compute the same thing and get the same gradient, so they never become different.**
2. Which initialisation goes with ReLU? **He initialisation (Xavier/Glorot for sigmoid and tanh).**
3. Training loss falls, validation loss rises. Diagnosis and fix? **Overfitting: regularisation, dropout, more data, early stopping.**
4. Where do the mean and standard deviation for standardising come from? **The training set only; reuse them for validation and test.**
5. How is prediction different from training? **Only the forward pass; no loss, no backward pass, no update.**

🗨 IN ONE BREATH

- Start random (He for ReLU, Xavier for sigmoid/tanh), biases zero. Never all zeros.
- $\eta \in \{0.001, 0.01, 0.1\}$ or Adam; $B \in \{32, \dots, 256\}$; 50–200 epochs with early stopping.
- Standardise inputs with training statistics; fill missing values; one-hot or ordinal encoding.
- Watch training and validation loss; read the symptom table; overfit a small batch first.
- Prediction = forward pass; regression reads the value, binary thresholds at 0.5, multi-class takes the arg max.

Losses, their slopes, and what comes next

Softmax with a worked problem, then every loss you have met with its derivative, side by side. Then the takeaways of both decks, and where Nova goes from here.

On your slides: **Numerical deck, slides 47–49, 57–61, 63–80** · Softmax definition, notation, softmax + cross-entropy · Softmax numerical problem · MSE, MAE, BCE, cross-entropy: definitions, derivatives, examples · Comparison tables · Key takeaways · Playground · **Module 5, slides 63–66** · Key takeaways · The power of deep learning · What next · To become proficient

STORY

Nova hears its name now. Asha wants more: four answers, not two. Name, question, door slam, silence. Four classes means a softmax output with cross-entropy, so she works one softmax problem by hand to be sure of the gradient. Then she makes a card for the wall: every loss she has used, and the slope of each, because the slopes are what backprop eats. Ravi reads the card. "So every loss ends up as 'prediction minus target' in some form?" "Almost every one," says Asha. "That is not an accident."

BIG IDEA

Backpropagation needs one thing from the loss: its slope with respect to the prediction. Each loss has a simple one. MSE: $\hat{y} - y$. MAE: $\text{sign}(\hat{y} - y)$. BCE: $(\hat{y} - y) / (\hat{y}(1 - \hat{y}))$. Cross-entropy: $-y_j / \hat{y}_j$. And when cross-entropy is paired with softmax, the slope with respect to the logits collapses to $\hat{y}_j - y_j$: prediction minus target, again.

THINK OF IT LIKE...

You are walking home in fog. The loss tells you how far from home you are. Its slope tells you which way to walk and how urgently. A steep slope (a confident wrong answer under BCE) says "turn around, fast". A flat slope says "small adjustment". The whole of training is: read the slope, take a step, read it again.

Softmax

Softmax turns a vector of C real numbers (the **logits** z) into a probability distribution:

$$\text{softmax}(z)_j = \hat{y}_j = \frac{e^{z_j}}{\sum_{k=1}^C e^{z_k}}, \quad j = 1, \dots, C$$

Four properties from your slide. Every output is in $(0, 1)$. The outputs sum to 1. It is monotonic: a bigger z_j gives a bigger $\hat{y}_j$. And it is translation invariant: $\text{softmax}(z + c) = \text{softmax}(z)$ for any constant c , because $e^{z_j+c} = e^c e^{z_j}$ and the e^c cancels top and bottom. Use case: the output layer of multi-class classification.

Your slide's example, $z = (1.0, 2.0, 3.0)$: the exponentials are 2.718, 7.389, 20.086, sum 30.193, so $\hat{y} = (2.718, 7.389, 20.086) / 30.193 = (0.090, 0.245, 0.665)$, which adds to 1.0. The largest logit gets the largest probability, but not all of it.

Softmax + cross-entropy: the clean gradient

Cross-entropy with a one-hot target, true class k : $\ell = -\sum_c y_c \log \hat{y}_c = -\log \hat{y}_k$. We want the slope with respect to each logit z_j , because that is $\delta^{(L)}$ in Chapter 4.

Σ MATHS, STEP BY STEP (YOUR SLIDE'S EQUATIONS 8–12)

PLAIN MATHS

Chain rule through every output $\hat{y}_i$:

$$\frac{\partial \ell}{\partial z_j} = \sum_{i=1}^C \frac{\partial \ell}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial z_j}$$

Only $\hat{y}_k$ is in the loss: $\partial \ell / \partial \hat{y}_i = -y_i / \hat{y}_i$, which is $-1/\hat{y}_k$ for $i = k$ and 0 otherwise. So only the $i = k$ term survives:

$$\frac{\partial \ell}{\partial z_j} = -\frac{1}{\hat{y}_k} \cdot \frac{\partial \hat{y}_k}{\partial z_j}$$

The slope of softmax is $\partial \hat{y}_k / \partial z_j = \hat{y}_k (\delta_{kj} - \hat{y}_j)$, where δ_{kj} is 1 if $k = j$ and 0 otherwise (the Kronecker delta, not an error signal).

Substitute; the $\hat{y}_k$ cancels:

$$\frac{\partial \ell}{\partial z_j} = -(\delta_{kj} - \hat{y}_j) = \hat{y}_j - \delta_{kj} = \hat{y}_j - y_j$$

Prediction minus target, for every class j .

VECTOR AND BATCH FORM

All C logits at once:

$$\frac{\partial \ell}{\partial \mathbf{z}} = \hat{\mathbf{y}} - \mathbf{y}$$

For a mini-batch of B rows, averaged:

$$\delta^{(L)} = \frac{\partial J}{\partial \mathbf{Z}^{(L)}} = \frac{1}{B} (\hat{\mathbf{Y}} - \mathbf{Y}) \in \mathbb{R}^{B \times C}$$

This is the third of Chapter 4's three pairings. Sigmoid+BCE gave the same form for one output; softmax+CE gives it for C outputs. That is why the output layer's error signal never needs the messy loss slope on its own: the activation was chosen to cancel it.

WORKED EXAMPLE: THE SOFTMAX PROBLEM FROM THE NUMERICAL DECK

Logits $\mathbf{z} = (2.0, 1.0, 0.1, -1.0)$, true label $\mathbf{y} = (0, 1, 0, 0)$ (class 2). Compute the softmax, the cross-entropy loss, and the gradient with respect to the logits.

- 1 For numerical stability, subtract the maximum first: $\mathbf{z} - \max(\mathbf{z}) = (2.0 - 2.0, 1.0 - 2.0, 0.1 - 2.0, -1.0 - 2.0) = (0.0, -1.0, -1.9, -3.0)$.

allowed by translation invariance; it stops e^z from overflowing for big logits

- 2 Exponentials: $e^0 = 1.000$, $e^{-1.0} = 0.368$, $e^{-1.9} = 0.150$, $e^{-3.0} = 0.050$.

- 3 Sum: $1.000 + 0.368 + 0.150 + 0.050 = 1.568$.

- 4 Softmax: $(1.000, 0.368, 0.150, 0.050) / 1.568 = (0.638, 0.235, 0.096, 0.032)$; they add to $1.001 \approx 1$.

- 5 Loss: only the true class counts: $L = -y_2 \log \hat{y}_2 = -\log(0.235) = 1.447$.

the network gave the true class only 23.5%, so the loss is high

- 6 Gradient: $\partial L / \partial \mathbf{z} = \hat{\mathbf{y}} - \mathbf{y} = (0.638 - 0, 0.235 - 1, 0.096 - 0, 0.032 - 0) = (0.638, -0.765, 0.096, 0.032)$.

negative only for the true class: push its logit up, push all the others down

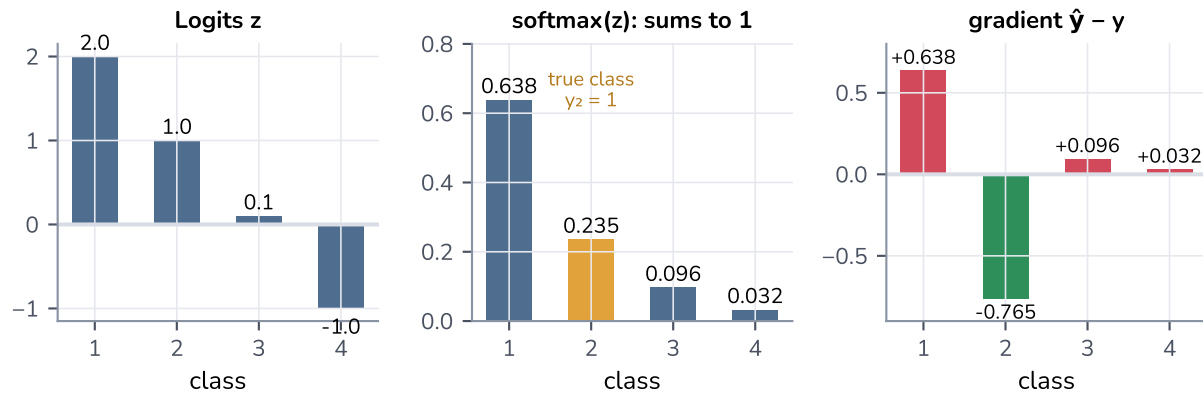


Figure 8.1 Left: the logits. Middle: softmax makes probabilities that sum to 1; the true class (orange) got only 0.235. Right: the gradient $\hat{y} - y$, negative for the true class, positive for the rest, and summing to zero: raising one logit lowers the others.

The four losses and their slopes

MSE (REGRESSION)

$\ell(\hat{y}, y) = \frac{1}{2}(\hat{y} - y)^2$. Always ≥ 0 ; convex for linear models; differentiable everywhere; penalises large errors quadratically; the $\frac{1}{2}$ is there to make the derivative clean. Derivation: let $u = \hat{y} - y$, so $\ell = \frac{1}{2}u^2$; chain rule gives $\partial\ell/\partial\hat{y} = \frac{1}{2} \cdot 2u \cdot 1 = u$:

$$\frac{\partial\ell}{\partial\hat{y}} = \hat{y} - y$$

Examples: $y = 3, \hat{y} = 5: \ell = \frac{1}{2}(2)^2 = 2$, slope = $5 - 3 = 2$. $y = 8, \hat{y} = 5: \ell = \frac{1}{2}(-3)^2 = 4.5$, slope = $5 - 8 = -3$. The negative slope says: increase $\hat{y}$.

MAE (REGRESSION WITH OUTLIERS)

$\ell(\hat{y}, y) = |\hat{y} - y|$. Always ≥ 0 ; less sensitive to outliers than MSE; not differentiable at $\hat{y} = y$; linear penalty. The derivative of $|x|$ is $+1$ for $x > 0$, -1 for $x < 0$, undefined at 0 (set to 0 by convention):

$$\frac{\partial\ell}{\partial\hat{y}} = \text{sign}(\hat{y} - y)$$

Examples: $y = 3, \hat{y} = 5: \ell = 2$, slope $+1$. $y = 8, \hat{y} = 5: \ell = 3$, slope -1 . Compare: the MAE slope is always ± 1 ; the MSE slope grows with the size of the error.

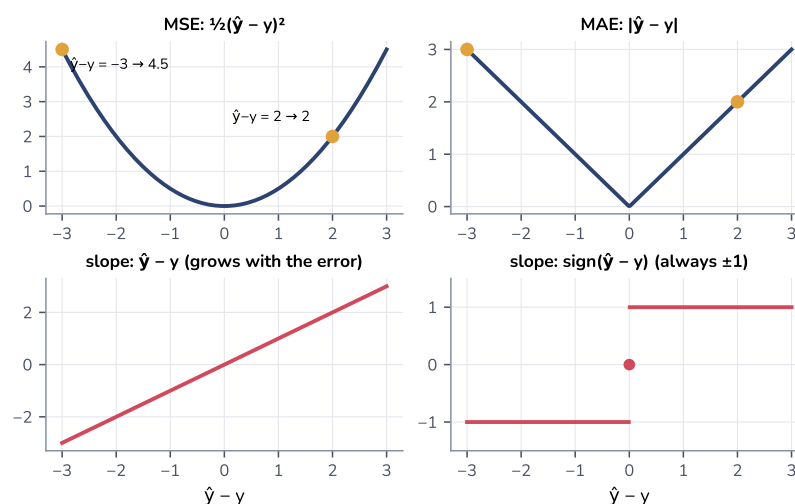


Figure 8.2 MSE and MAE against the error $\hat{y} - y$, slopes underneath. MSE's slope grows with the error (big error, big push); MAE's is a flat ± 1 . That is why MAE shrugs off outliers.

BCE (BINARY CLASSIFICATION)

$\ell(\hat{y}, y) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$ with $y \in \{0, 1\}$, $\hat{y} \in (0, 1)$. It measures how far the predicted probability is from the true one, is always ≥ 0 , and heavily penalises confident wrong predictions. Because y is 0 or 1, only one term is ever alive. If $y = 1$, $\ell = -\log \hat{y}$, so we want $\hat{y} \rightarrow 1$. If $y = 0$, $\ell = -\log(1 - \hat{y})$, so we want $\hat{y} \rightarrow 0$.

MATHS, STEP BY STEP: THE BCE SLOPE

PLAIN MATHS

Differentiate each term, using $\frac{d}{dx} \log x = \frac{1}{x}$ and the chain rule (the inner derivative of $1 - \hat{y}$ is -1):

$$\frac{\partial \ell}{\partial \hat{y}} = -\frac{y}{\hat{y}} - (1 - y) \cdot \frac{1}{1 - \hat{y}} \cdot (-1) = -\frac{y}{\hat{y}} + \frac{1 - y}{1 - \hat{y}}$$

Put over a common denominator:

$$\frac{-y(1 - \hat{y}) + (1 - y)\hat{y}}{\hat{y}(1 - \hat{y})} = \frac{\hat{y}(1 - \hat{y}) - y + y\hat{y} + \hat{y} - y\hat{y}}{\hat{y}(1 - \hat{y})} = \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})}$$

THE THREE EXAMPLES

$y = 1, \hat{y} = 0.9$: $\ell = -\log 0.9 = 0.105$; slope = $\frac{0.9 - 1}{0.9 \times 0.1} = \frac{-0.1}{0.09} = -1.11$.

$y = 1, \hat{y} = 0.1$: $\ell = -\log 0.1 = 2.303$; slope = $\frac{-0.9}{0.1 \times 0.9} = \frac{-0.9}{0.09} = -10$.

$y = 0, \hat{y} = 0.9$: $\ell = -\log 0.1 = 2.303$; slope = $\frac{0.9}{0.9 \times 0.1} = \frac{0.9}{0.09} = 10$.

Confident and wrong: loss 2.3 and a slope of size 10. Confident and right: loss 0.1 and a slope of 1.1. The wrong answer is pushed nine times harder.

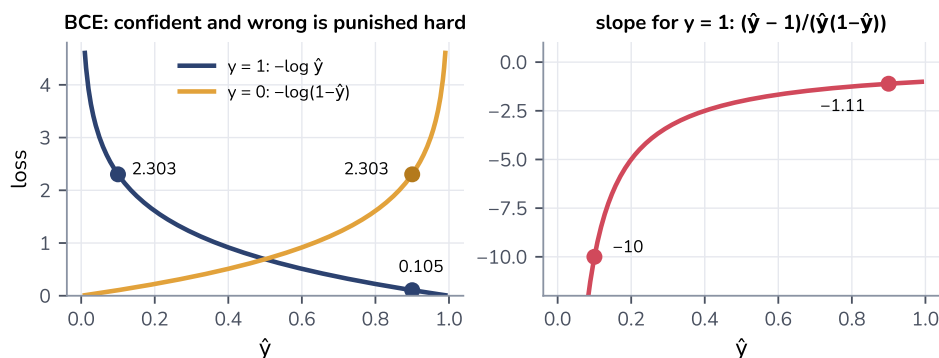


Figure 8.3 Left: BCE for $y = 1$ (blue) and $y = 0$ (orange), with the three examples marked. Right: the slope for $y = 1$; notice it is -1.11 at $\hat{y} = 0.9$ and -10 at $\hat{y} = 0.1$, and it goes to $-\infty$ as $\hat{y} \rightarrow 0$.

CROSS-ENTROPY (MULTI-CLASS)

$\ell(\hat{\mathbf{y}}, \mathbf{y}) = -\sum_{c=1}^C y_c \log \hat{y}_c$, with $\mathbf{y}$ one-hot and $\hat{\mathbf{y}}$ a probability vector (from softmax). Only the $c = j$ term depends on $\hat{y}_j$, so

$$\frac{\partial \ell}{\partial \hat{y}_j} = -\frac{y_j}{\hat{y}_j}, \quad \nabla_{\hat{\mathbf{y}}} \ell = \left(-\frac{y_1}{\hat{y}_1}, \dots, -\frac{y_C}{\hat{y}_C} \right)^\top$$

Since $\mathbf{y}$ is one-hot with $y_k = 1$: the true class gets $-1/\hat{y}_k$ and every other class gets 0. This is the slope with respect to the softmax *outputs*. Combine it with the softmax slope (the ladder above) and it becomes the much simpler $\hat{y}_j - y_j$ with respect to the logits.

Two comparison tables

Property	Sigmoid	Tanh	Softmax
Formula	$1/(1 + e^{-z})$	$(e^z - e^{-z})/(e^z + e^{-z})$	$e^{z_j} / \sum_k e^{z_k}$
Range	$(0, 1)$	$(-1, 1)$	$(0, 1)$
Zero-centred?	no	yes	no
Derivative	$\sigma(1 - \sigma)$	$1 - \tanh^2$	complex ($\hat{y}_k(\delta_{kj} - \hat{y}_j)$)
Max derivative	0.25	1.0	varies
Output sum	n/a	n/a	1.0
Use case	binary output	hidden layers	multi-class output

Loss	Formula	Derivative w.r.t. $\hat{y}$	Use
MSE	$\frac{1}{2}(\hat{y} - y)^2$	$\hat{y} - y$	regression
MAE	$ \hat{y} - y $	$\text{sign}(\hat{y} - y)$	regression with outliers
BCE	$-[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$	$\frac{\hat{y} - y}{\hat{y}(1 - \hat{y})}$	binary classification
Cross-entropy	$-\sum_c y_c \log \hat{y}_c$	$-y_j/\hat{y}_j$ (with softmax: $\hat{y}_j - y_j$ w.r.t. z_j)	multi-class

! CAREFUL

Always compute softmax with the maximum subtracted (or the log-sum-exp trick): e^{1000} overflows, e^0 does not, and translation invariance guarantees the same answer. Also, the deck's loss 1.447 comes from the rounded 0.235; unrounded, it is 1.449. Show your rounding and either is accepted.

Key takeaways of both decks

Module 5

Non-linearity is essential: without it layers collapse to a linear model. A DFNN = stacked layers with non-linear activations, learning a hierarchy of features. Four components: data, model, objective, learning (backprop + SGD). Backprop gets every gradient from the chain rule. Depth gives hierarchy and expressiveness; width gives capacity but needs data. Initialisation, normalisation and regularisation are critical. ReLU for hidden layers; the task sets the output activation.

Numerical deck

MSE: quadratic penalty, linear derivative. MAE: robust to outliers, derivative ± 1 . BCE: binary, punishes confident errors. Cross-entropy: multi-class, only the true class's probability matters. Sigmoid $\rightarrow (0, 1)$; tanh $\rightarrow (-1, 1)$, zero-centred; softmax $\rightarrow$ a probability distribution. Regression: MSE or MAE. Binary: sigmoid + BCE. Multi-class: softmax + cross-entropy.

The power of deep learning, and what comes next

Your slide draws the whole idea in one line: raw input $\rightarrow$ features $\rightarrow$ patterns $\rightarrow$ concepts $\rightarrow$ prediction, each arrow a weight matrix, all learned from data by gradient descent. It has changed computer vision, language, speech, game playing (Go, chess, StarCraft) and science (protein

folding, drug discovery).

What next. Specialised architectures: *convolutional* networks for images (Book 6), *recurrent* networks for sequences, *transformers* for attention (GPT, BERT), *graph* networks. Advanced topics: optimisers (Adam, RMSprop, AdaGrad), dropout and batch normalisation, transfer learning, interpretability, efficient inference. Your slide's last line: **DFNNs are the foundation — master them first.**

To become proficient. Implement from scratch (forward pass, backprop, gradient checking; train on XOR and MNIST). Use frameworks well (PyTorch, TensorFlow, JAX do backprop, but the maths makes debugging easier). Experiment and iterate; track experiments; learn from failures. Study successful architectures. And try **playground.tensorflow.org**: pick the XOR dataset, add a hidden layer, and watch the fold from Chapter 1 happen live.

🔪 STORY: WHAT NOVA STILL CANNOT DO

Nova can hear. It wakes to its name, knows a question from a statement, and ignores the door. Asha puts the marker down and picks up a small camera module. "Now I want you to see me." Ravi looks at the 784 numbers in one tiny image and at the parameter count from Chapter 3. "That is going to need a different kind of layer." It is. That is Book 6.

📄 ON YOUR SLIDES

Numerical deck: slides 47–49 softmax (definition, notation and the (1, 2, 3) example, the softmax + cross-entropy derivation (7)–(12)); 57–61 the softmax problem; 64–66 MSE; 67–69 MAE; 70–73 BCE; 74–76 cross-entropy; 78–79 the comparison tables; 80 key takeaways; 81 the playground. Module 5: slides 63–66 key takeaways, the power of deep learning, what next, to become proficient.

✅ CHECK YOURSELF

- Logits $z = (2, 0)$, true class 2: softmax, loss, gradient? $(0.881, 0.119)$; $-\log 0.119 = 2.13$; $(0.881, -0.881)$.
- What is the gradient of cross-entropy with respect to the logits, after softmax? $\hat{y}_j - y_j$.
- MSE at $y = 8, \hat{y} = 5$: loss and slope? **4.5 and -3.**
- BCE at $y = 1, \hat{y} = 0.1$: loss and slope? **2.303 and -10.**
- Which loss goes with softmax, and which with sigmoid? **Cross-entropy with softmax; BCE with sigmoid.**

🔵 IN ONE BREATH

- Softmax: $e^{z_j} / \sum_k e^{z_k}$; outputs in $(0, 1)$, sum to 1, shift-invariant. Subtract the max for stability.
- Softmax + cross-entropy: $\partial \ell / \partial z_j = \hat{y}_j - y_j$. The problem: softmax $(0.638, 0.235, 0.096, 0.032)$, loss 1.447.
- Slopes: MSE $\hat{y} - y$; MAE $\text{sign}(\hat{y} - y)$; BCE $(\hat{y} - y) / (\hat{y}(1 - \hat{y}))$; CE $-y_j / \hat{y}_j$.
- Regression → MSE/MAE; binary → sigmoid + BCE; multi-class → softmax + CE.

Where you are in the series

Six small books, one story. Asha builds Nova, a robot that must learn to hear its name — and along the way she meets every idea in the Deep Neural Network course, in the order the course teaches them.

BOOK 1

Teaching a Machine to Listen

What deep learning is, and the four parts of every learning problem

BOOK 2

One Tiny Brain Cell

The artificial neuron, the perceptron, logic gates and the XOR wall

BOOK 3

Guessing How Much

A linear neuron for regression, squared loss and gradient descent

BOOK 4

Guessing Which One

Sigmoid and softmax neurons, cross-entropy, SGD and mini-batches

BOOK 5

A Real Brain

Hidden layers, activations, backpropagation by hand, and design

BOOK 6

Eyes for the Machine

Convolution, feature maps, pooling, famous CNNs and transfer learning

Next: **Book 6, Eyes for the Machine** — Nova can hear. Now Asha gives Nova a camera: images, convolution, feature maps, and the famous CNNs.